

Optimal and Automated Meta-path Selection for Heterogeneous Graph Neural Networks

Author:
Nasib Huseynzade
1748504

Supervisor:
Nourhan Ahmed

1st October 2026

Thesis submited for

Master of Science in Data Analytics

Wirtschaftsinformatik und Maschinelles Lernen
Stiftung Universität Hildesheim
Universitatsplätz 1, 31141 Hildesheim

Abstract

Heterogeneous Graph Neural Networks have attracted increasing attention due to their ability to model complex systems involving multiple types of nodes and relations. In this context, metapaths play a crucial role by capturing high-level semantic relationships and guiding information propagation in heterogeneous graphs. However, recent efficient models such as SeHGNN use all metapaths up to a maximum hop that is fixed for each dataset, without considering which of them are relevant to the task. Moreover, the number of possible metapaths grows rapidly with the metapath length, and since the cost of semantic fusion grows quadratically with the number of metapaths, redundant or uninformative metapaths increase the computation without improving the prediction. Motivated by these challenges, I propose a task-aware and automatic metapath selection method that is applied as a pre-processing step before SeHGNN, without changing its architecture. The method scores each metapath by the conditional information gain it brings about the task labels, that is, by how much it improves the prediction of the labels given the metapaths already selected, and adds metapaths greedily until this gain vanishes. In this way, redundant metapaths are excluded and the depth of the selected metapaths is determined from the data. The proposed method is evaluated on three real-world heterogeneous graph datasets, where it keeps only a few of the candidate metapaths. It matches the accuracy of SeHGNN and other newest models, while reducing the number of parameters by 66–74% and making each training epoch 2.7–3.4 times faster with the largest candidate sets. Code is available at https://github.com/nasibhuseynzade/IG-SeHGNN

Keywords: Heterogeneous Graph Neural Networks, Metapath,

Contents

1 Introduction **1**

1.1 Background . . . 1
1.2 Problem Statement . . . 3
1.3 Proposed Approach and Contributions . . . 4

2 Preliminaries **6**

2.1 Heterogeneous Graphs . . . 6
2.2 Metapaths . . . 6
2.3 Problem Definition . . . 7

3 Related Work **8**

3.1 Heterogeneous Graphs, Metapaths, and Two Families of HGNNs 8
3.2 Node-Level Aggregation versus Semantic-Level Fusion . . . 9
3.3 SeHGNN as the Backbone Model . . . 11
3.4 Metapath Search and Selection . . . 11
3.5 Research Gap . . . 14
3.6 Positioning of the Proposed Method . . . 16

4 Methodology **19**

4.1 Overview . . . 19
4.2 Metapath channels . . . 20
The SeHGNN backbone . . . 22
4.3 Candidate generation . . . 23
4.4 Selection criterion: conditional information gain . . . 24
4.5 Estimating the information gain with a probe . . . 25
The bag-of-channels probe . . . 25
Held-out score . . . 26
The estimated information gain . . . 27
4.6 Greedy selection with automatic depth . . . 27
Forward selection . . . 27
Stopping rule and output . . . 28

Automatic depth . . . 28
Properties and limits of the greedy procedure . . . 28
4.7 Integration with SeHGNN and computational effect . . . 29
Overall procedure . . . 29
Effect on the number of parameters . . . 29
Effect on the training time . . . 29
Cost of the selection itself . . . 29
4.8 Relation to MP-GNN and SeHGNN . . . 31
Relation to MP-GNN . . . 31
Relation to SeHGNN . . . 32
4.9 Implementation details . . . 33
4.10 Summary . . . 33

5 Experimental Setup **35**
5.1 Datasets . . . 35
5.2 Baselines . . . 36
5.3 Evaluation Metrics . . . 38
5.4 Hyper-parameters and Implementation . . . 39
5.5 Experiments . . . 40

6 Results **44**
6.1 Reproduction of SeHGNN . . . 44
6.2 Classification accuracy . . . 44
6.3 Computational cost . . . 46
6.4 Selected metapaths . . . 47
6.5 Summary . . . 49

7 Ablation Studies **50**
7.1 Stopping threshold and minimum number of metapaths . . . 51
7.2 Selection criterion . . . 53
7.3 Cost of the selection . . . 55
7.4 Summary . . . 56

8 Conclusion **57**
8.1 Summary . . . 57
8.2 Main findings . . . 57
8.3 Limitations . . . 58
8.4 Future work . . . 59

9 Appendix **60**
9.1 Information-theoretic definitions 60
9.2 A worked example 61
9.3 Cross-entropy as an upper bound of the conditional entropy . 62
9.4 Multi-label probe 63
9.5 Parameter count and selection cost 63
9.6 Implementation details of the SeHGNN backbone 64
9.7 Scoring function of MP-GNN 65

List of Figures

3.1 The shown architecture of the Original SeHGNN model. . . . 11

4.1 Overview of IG-SeHGNN. Left: a heterogeneous graph with a target node. Middle: task-aware automatic metapath selection; all candidate metapaths up to the hop budget are reduced to a small set of selected metapaths. Right: the unchanged SeHGNN backbone (Section 4.2); feature projection of each selected channel and transformer-based semantic fusion, followed by concatenation and the downstream task. 21

6.1 Feature-metapath candidate pool of IMDB (25 metapaths of up to four hops) and the five metapaths kept by IG-SeHGNN in the main configuration. 49

7.1 Greedy selection paths: cross-validated score after each greedy step and the estimated information gain of the first steps; the metapaths kept with $\varepsilon = 0.005$, $m = 1$ are highlighted. 52

List of Tables

3.1 Node-level aggregation and semantic-level fusion in representative HGNNs. 17
3.2 Automatic metapath selection methods: selection signal and the stage at which selection takes place. 18

4.1 Size of the candidate pools for $L = L_Y = 4$: number of feature metapaths per hop, total number of feature metapaths $|\Phi_X|$, number of label metapaths $|\Phi_Y|$ and total number of channels k. 24
4.2 Comparison of MP-GNN and IG-SeHGNN. 32
4.3 Hyper-parameters of the selection step. 33

5.1 Statistics of the datasets as reported in [8, 11]. The numbers refer to the datasets as provided by HGB; the pre-processing of ACM is described in Section 5.1. 36
5.2 Compared methods. "Label prop." states whether the reported results use label propagation, and "Source of results" whether the results are quoted from the cited table or come from our own runs; "–" means that this is not stated in the cited source. L and L_Y are the hop budgets of feature and label metapaths. 42
5.3 SeHGNN hyper-parameters that differ between the datasets, taken from the published configuration of the official implementation. All other SeHGNN hyper-parameters are the same on all datasets (Section 5.4); the selection hyper-parameters of IG-SeHGNN are listed in Table 4.3. 43

6.1 Reproduction of SeHGNN (paper setting): test F1 (%) reported in [12, Table 3] 45

6.2 Test F1 (%) on the HGB datasets. Baseline results are quoted from [12, Table 3] (first nine rows), [8, Table 1] and [1, Table 3]; LMSPS is evaluated without label propagation (Section 5.2). IG-SeHGNN: main configuration, Bold: best value per column. 45
6.3 Computational cost (same machine). For DBLP, SeHGNN is reported with the hop budget of its paper setting (2) and with the hop budget of four from which IG-SeHGNN selects; on ACM and IMDB the paper setting already uses a hop budget of four. The last three columns compare IG-SeHGNN with the SeHGNN configuration of the same row. 46
6.4 Feature metapaths used by SeHGNN (paper setting) and selected by IG-SeHGNN (main configuration). 48

7.1 DBLP. Reference: SeHGNN with the hop budget of its paper setting, $k = 9$, 10.8M parameters, 0.359 s per epoch, 95.35 / 94.97. 51
7.2 ACM. Reference: SeHGNN, $k = 61$, 88.5M parameters, 2.419 s per epoch, 93.86 / 93.93. 51
7.3 IMDB. Reference: SeHGNN, $k = 37$, 83.7M parameters, 3.119 s per epoch, 68.91 / 66.23. 52
7.4 Compared selection methods. 54
7.5 DBLP: test Micro-F1 / Macro-F1 (%). 55
7.6 IMDB: test Micro-F1 / Macro-F1 (%). Reference: SeHGNN with all 25 feature metapaths, 68.91 / 66.23. 55

9.1 Toy example with eight labelled nodes and three candidate channels. 62

Chapter 1

Introduction

1.1 Background

In many scenarios the real world data is graph structured, such as social networks, citation networks or the World Wide Web [7]. The structure of such graphs is non-Euclidean [5] and conventional neural networks are not very successful on such data, while graph neural networks (GNNs) are very successful. GNNs are a way of extending a deep neural network to be able to handle graph-structured data [7]. The difference between a GNN and a standard NN is that, a GNN learns the node representations based on the node features and the graph structure [7]. The majority of them are based on a neighborhood aggregation (i. e., message-passing) scheme, where the representation of each node is recursively aggregated from its neighbors features [3, 12]. For example, graph convolutional networks (GCNs) are a generalization of convolution to graphs [7]. GCN aggregates the features of each node's 1-hop neighbors in each layer; GraphSAGE introduces mini-batch training and neighbor sampling to enable scaling to large graphs; and GAT employs attention to focus on the most important neighbours [12]. GNNs have been used in topics like traffic-flow prediction, social media and recommender systems [11].

There are two types of graphs: homogenous and heterogeneous. Most of the successful GNNs, including GCN [11] and GAT [7, 13] are developed for homogeneous graphs, where all nodes are of the same type and all edges are of the same type. A heterogeneous graph is a graph with more than one kind of vertices or more than one kind of edges, which is referred to as a heterogeneous information network (HIN) [7]. In practice, many complex systems can be naturally represented as heterogeneous graph, where the entity and the relation types are presented as different node and edge types [12]. The

citation network, ACM, for instance, has paper, author and subject nodes and the relationships include "author writes paper" and "paper cites paper" and "paper is a member of subject [12]". However, there are extra difficulties with heterogeneity, compared to homogeneous graphs [13]. First, the structure around a node differs based on the relation under consideration: in an academic network, the neighbours of a paper may be its authors (with the write relation) or its terms (with the contain relation) [13]. Secondly, different type of nodes may have features of same name with different meanings (in different feature spaces [7, 13]).

However, due to this heterogeneity, unlike homogeneous graphs, which carry only structural information, the GNNs cannot be directly used for heterogeneous graphs. Making a homogeneous graph with a unique treatment of data in all different relations is not the ideal choice, as different relations can contain vastly different semantic information [10]; inexpert message passing that does not consider node and edge types at all, can lead to sub-optimal performance [3]. Heterogeneous graph neural networks (HGNNs) thus have been proposed to incorporate both structure and semantic information into node representations, which are applied to social network analysis, recommendation and knowledge graph inference, among others [12].

One of the important means of capturing semantics in a heterogeneous graph is metapath, which is a composition of node types and edge types [7, 13]. There are different semantics represented by different metapaths. In ACM, the two-hop metapath Author–Paper–Author (APA) is considered as co-author relation, and the four-hop metapath Author–Paper–Subject–Paper–Author (APSPA) is considered as two authors on the same subject [12]. Generally, HGNNs can be categorized as metapath based and metapath free approaches [11, 12]. The key idea of metapath-based methods is that they first combine the neighbor features within the scope of each metapath to get a semantic vector for each metapath and then fuse the semantic vectors to get the final embedding. The metapath-free methods combine messages from the local neighborhood of nodes like with ordinary GNNs, but with additional modules, including attention, to capture node and edge types [12]. It has been reported that metapath-based HGNNs seem to have more potential for large-scale data sets, since they selectively aggregate neighbors along metapaths.[8]

Metapath-based HGNNs rely on the selection of metapaths to specify which neighbours are aggregated, but it is not easy to select the metapaths [8]. Early models e.g., HAN use manually designed metapaths [3] and the authors of HAN admit that it is an open problem to choose the most meaningful metapaths for a specific task [7]. However, the manual design of metapaths needs domain knowledge [3, 13] and it is not possible to manually choose

informative metapaths for each new graph because of its associated trial and error [6]. Manual pre-selection is also subjective and will miss out relevant metapaths [2]. Furthermore, a metapath can only be useful in the context of a task. Not all heterogenous graph semantic information is beneficial to a specific downstream task: for example, when predicting the research area an author belongs to, the institutions he works for are unrelated and may introduce noise [3]. Similarly, in the case of recognizing the genre of a movie, the metapath Movie–Actor–Movie is more significant than Movie–Year–Movie [7].

1.2 Problem Statement

Recently, a HGNN designed for both accuracy and training speed, called SeHGNN [12] has been developed based on metapath. It does not however choose its metapaths based upon the task. The metapath set is derived from the propagation step and is the collection of all metapaths that start at the source node type and end at the target node type within a maximum number of hops [8, 11]. There are two implications of this. Initially, the number of metapaths increases exponentially as the maximum hop is increased [8]. Second, the fixed price tag of the metapath is not limited to pre-processing. Each metapath is given its own projection MLP, and the number of parameters increases with the number of metapaths, and the time complexity of semantic fusion has a term that is quadratic in the number of metapaths [12]. While reducing the maximum hop is a reasonable workaround, it is not a corrective as the SeHGNN study itself concluded that the longer the metapath the better, and that the metapaths with different semantics are more beneficial at improving the performance [12].

Furthermore, many of the listed metapaths are of little influence. When analyzing on DBLP/ACM, Li et al. [8] found that a few metapaths contribute the major work and some metapaths even have the negative effect. This degradation shows that attention is not good at handling noisy metapaths [8] and SeHGNN combines metapaths with attention. Metapaths can also be redundant, that is, the same sequence of nodes in the same order from one node to another. These redundant metapaths make the computation more expensive and it also hinders in the learning process [11].

Following Khan and Aboulnaga [9], GNNs can be divided into two groups. The first group learns metapath importance inside model. Here, HAN [7] assigns weights to a set of metapaths with semantic-level attention, GraphMSE [6] samples the metapaths and nodes and performs metapath selection with attention, and GTN, HGT and others learn metapaths implicitly with at-

tention [3]. The second group considers selection as a problem by itself. DiffMG [3] performs differentiable neural architecture search for meta graphs, AMP-HGNAS [1] performs a unified neural architecture search to search for combination of metapaths and aggregation strategies, and LMSPS [8] searches for useful metapaths with a super-net and then re-trains the model with only searched metapaths. A selection is only available if all these approaches must build a model, which is called a search network, beforehand. Moreover, LMSPS maintains a constant number of metapaths - say 30 - on every dataset [8] and has been criticized for ignoring interdependencies between metapaths [1].

MP-GNN [10] greedily builds metapaths that satisfy a criterion computing the relational information gain of continuing the metapath prefix, and these metapaths are the layers of a dedicated MP-GNN architecture; it aims at finding metapaths whose discovery is a useful feature for the label. For relational deep learning, MetaSieve [9] scans candidate extensions by their mutual information with its occurrence statistics and the task label. It is not dependent upon GNN parameters, and can serve as a pre-processing operation for various GNN backbones [9]. Authors of the extension talk of its usefulness as reducing the validation error it entails in addition to the statistics already available, and they use mutual information as a quick and dirty measure since the cost of training the appropriate models would be prohibitive [9]. However, a score based on dependence of each candidate on the label alone does not reflect the information given by previously chosen metapaths. On the other hand, metapath construction designed for redundancy removal, like PM-HGNN [11], merges metapaths with high structural similarity, and therefore cleanses structural redundancy, but does not account for the labels. From the methods mentioned above, none of them choose the metapaths according to label information of the metapaths given the selected metapaths, and at the same time, without changing the downstream model (such as SeHGNN).

1.3 Proposed Approach and Contributions

This thesis proposes a task-aware and automatic metapath selection method that is applied as a pre-processing step before SeHGNN, without changing its architecture. The method scores each candidate metapath by the conditional information gain it brings about the task labels, that is, by how much it improves the prediction of the labels given the metapaths already selected. Metapaths are added greedily until this gain vanishes. In this way, redundant metapaths are excluded, and the metapath length is determined

from the data instead of being fixed in advance. The criterion follows the incremental Max-Dependency scheme from feature selection, in which each step adds the feature that yields the largest increase in the mutual information between the selected set and the target class [4]. Selecting metapaths before training suits SeHGNN: Li et al. [8] showed that replacing SeHGNN's original metapath set with separately searched metapaths, while keeping all other settings unchanged, improved its micro-F1 on DBLP, IMDB, ACM and Freebase. Because SeHGNN computes one feature matrix, one projection MLP and one fusion input per metapath, a smaller metapath set reduces pre-processing, parameters and fusion cost at the same time.

The main contributions of this thesis are as follows:

1. A metapath scoring criterion based on the conditional information gain about the task labels, combined with a greedy selection procedure that stops when the gain vanishes.

2. The integration of this selection into SeHGNN as a pure pre-processing step, without changes to the model architecture.

3. Method is evaluated based on three real-world heterogeneous graph datasets and is shown to retain a small part of the candidate metapaths and gains competitive accuracy on 2 of them relative to SeHGNN, while reducing up to 78% of the number of parameters and up to 2.7 times of training time per epoch.

Chapter 2

Preliminaries

2.1 Heterogeneous Graphs

Following Ferrini et al. [10] and Yang et al. [12], let $G = (V, E, T^v, T^e)$ be a heterogeneous graph, where V and E denote the sets of nodes and edges, and T^v and T^e denote the sets of node types and edge types, respectively. Each node $v \in V$ is mapped to its node type $\phi(v) \in T^v$ by a node type mapping function $\phi: V \to T^v$, and each edge $e \in E$ is mapped to its edge type $\psi(e) \in T^e$ by an edge type mapping function $\psi: E \to T^e$ [12]. The graph is heterogeneous if $|T^v|+|T^e|> 2$ [7]; if $|T^v|= |T^e|= 1$, it reduces to a homogeneous graph [12]. The structure of G can be represented by one adjacency matrix A_r for each edge type $r \in T^e$ [12]. Nodes of different types may have features of different dimensions [6]; the features of all nodes of type c are collected in a matrix $X^c \in \mathbb{R}^{|V^c| \times d^c}$, where V^c is the set of nodes of type c and d^c is their feature dimension [12].

2.2 Metapaths

A metapath is defined as a sequence of node types $P = (c_1, c_2, \ldots, c_l)$ in which every two consecutive types are connected by an edge type; it represents a composite relation between nodes of type c_1 and nodes of type c_l [7, 12]. A sequence of node types is sufficient to describe the metapath because an edge type is determined by its source and target node types [3]. Following SeHGNN, c_1 is the type of the target nodes, and a metapath with l node types connects each target node to its $(l-1)$-hop neighborhood; it is therefore called an $(l-1)$-hop metapath [12]. Different metapaths reveal different semantics [7]. In a movie network, for example, Movie–Actor–Movie (MAM) expresses the co-actor relation, whereas Movie–Director–Movie (MDM) con-

nects movies that have the same director [7].

A metapath is a pattern on the level of types, and its occurrences in the graph are called metapath instances. Given P, a metapath instance is a node sequence $(v_1, v_2, \ldots, v_l)$ in G that follows P, that is, each v_i has type c_i and consecutive nodes are connected by edges of the corresponding types [12]. One metapath usually corresponds to many instances [8]. If an instance connects a target node v_1 with a node v_l, then v_l is a metapath-based neighbor of v_1 [12]. These neighbors can be obtained by multiplying the adjacency matrices along the metapath [7], and using a metapath therefore means aggregating information selectively from the neighbors on its instances [8].

2.3 Problem Definition

This thesis considers semi-supervised node classification on a target node type c, the setting in which HAN and SeHGNN are evaluated [7, 12]: the labels of a subset of the target nodes are known, and the labels of the remaining target nodes are to be predicted. Let Φ_L denote the candidate set of all metapaths related to the target type with at most L hops, which is the set that SeHGNN uses as a whole [8, 11]. Given G, a task $\mathcal{T}$ with loss $\mathcal{L}$ and the candidate set Φ_L, the objective is to learn representations $Z \in \mathbb{R}^{|V^c| \times d}$ of the target nodes that minimize $\mathcal{L}$, using a subset $\{P_k\} \subseteq \Phi_L$ of task-relevant metapaths that is discovered automatically instead of being fixed by hand [6, 8]. In this thesis, the subset is selected before training, and Z is then learned by SeHGNN using only the selected metapaths.

Chapter 3

Related Work

This chapter provides a literature review of the basis of the proposed metapath selection method. In Section 3.1, we have first introduced metapaths and the two primary families of heterogeneous graph neural networks (HGNNs), metapath based and metapath free methods. The difference between node-level aggregation and semantic-level fusion is summarized in Section 3.2 and is used as the basis to classify the representative models. The reasons for choosing the backbone model of this thesis, SeHGNN [12] is explained in Section 3.3. Section 3.4 surveys the current techniques in metapath search and selection and its difference between selecting a metapath during training and before training. At last, Section 3.5 designates the gap in the research and Section 3.6 sets the proposed method in the context of the gap.

3.1 Heterogeneous Graphs, Metapaths, and Two Families of HGNNs

Based on the different semantics adopted, HGNNs can be generally categorized into metapath-based [12, 8, 11] and metapath-free methods. While the metapath-free methods construct the semantic vectors by aggregating the features of the neighbors within a node's local scope and propagating them to the neighbors, then embed nodeand edge-type information by adding additional layers such as attention, the metapath-based ones do the same in the scope of each metapath. The demarcation between the two families is not consistent across all works. For example, RGCN is referenced as a metapath-based HGNN in [12] and as a metapath-free HGNN in [11] while the method SeHGNN is referred to as a metapath-based HGNN in [12] is discussed as a metapath-free HGNN in [5]. This thesis is along the lines of classification of [12].

HAN [7] and MAGNN are the representative metapath methods and the latest models are SeHGNN [12] and LMSPS [8] and PM-HGNN [11]. HAN collects the structural information on each metapath neighbor graph and finally fuses the outputs of different metapaths [12, 7] to separate different semantics. MAGNN also makes use of all nodes of a metapath instance instead of just the two end nodes [12]. Classical metapath-based models work on a tiny, manually given set of metapaths, such as {APA, APCPA, APTPA} on DBLP and {PAP, PSP} on ACM [7]. Thus, the quality of the predefined metapaths [11] has a large impact on their performance, and several studies question the assumption that a suitable set of metapaths exist a priori [6, 3, 1, 5].

The metapath-free methods do not require pre-defined metapaths. RSHN is designed to incorporate neighbor features and the use of embeddings based on an edge type that are derived by coarsening a line graph; HetSANN features a multi-layer GAT backbone with type-specific score functions; HGT uses a Transformer-based heterogeneous mutual attention with type-specific parameters; HGB (Simple-HGN) uses a multi-layer GAT backbone with learnable edge-type embeddings [12, 8]. Other models (e.g., GTN, HGT) have also been implicitly shown to learn metapaths by combining (fusing) types with attention [3]. This comes at a price of two. However, the inability to utilize semantic information from other semantically relevant neighbors can cause error in these models if they are directly aggregated over the neighborhood [11] (not all semantic information is relevant to a particular downstream problem [3]). Second, on large graphs like OGBN-MAG, all metapath-free methods ran out of memory in the experiments in [8] as a metapath-free model with l layers corresponds to an l-hop receptive field, which naturally grows exponentially as the number of layers increases. That explains why metapath-based HGNNs are considered as a more promising model for large-scale data sets [8].

3.2 Node-Level Aggregation versus Semantic-Level Fusion

Many HGNNs perform a two-step process that gives rise to representations of nodes. HAN has explicitly distinguished themselves with a hierarchical attention method, which is a node-level attention and a semantic-level attention [7]. Node-level attention gracefully learns the importance of neighbors of a node based on the metapath, and aggregates them to a semantic-specific node embedding; semantic-level attention learns the importance of

each metapath and fuses the semantic-specific embeddings to the final embedding by a weighted sum [7, 13]. Node-level aggregation thus retains information about structure within one metapath or relation while semantic-level fusion works across metapaths or relations and decides on the extent to which a semantic contributes to the task [7, 12].

SeHGNN [12] extends this distinction to other HGNNs, with the attention components be neighbour attention (among HGNNs in the same relation) and semantic attention (between HGNNs in different relations). In metapath based approaches (e.g., HAN), the two attentions are applied at different stages: neighbor aggregation and semantic fusion. Both are in one single attention over 1-hop neighbors computed with relation-specific embeddings; but can be separated by averaging or normalizing the attention values within each relation, as in HGB. With this separation, SeHGNN shows that deletion of semantic attention has a significant impact on the performance of HAN and HGB, but deletion of neighbor attention has no; it's worth noting that the neighbour attention is not necessary while the semantic attention is essential [12]. This observation is also significant in terms of efficiency, as the time for neighbor aggregation, based only on attention, consumes more than 85% of the total time in HAN and HGB [12].

Table 3.1 provides a summary of representative models that are used to implement the two levels. The concept of paying attention at both levels is used by HAN. MAGNN consists of an attention layer on the metapath instances within a metapath (intra-metapath aggregation) and learns the importance of different metapaths (inter-metapath aggregation) [13]. In these metapath-free models, neighbor and semantic weighting are not done in separate steps, as HetSANN and HGT leverage type-aware attention, which assign different weights for different neighbors during weighting, and HGB creates attention scores based on the 1-hop neighbors with relation-specific embeddings. SeHGNN doesn't use neighbor attention but instead uses a parameter-free mean aggregator and attention only for semantic fusion, which is in the form of Transformer [12] based. Moreover, PM-HGNN pre-computes a mean aggregation based on the global node similarity and optimizes the metapath weight during training based on the length and type of metapaths [11]. LMSPS takes it one step further: it concatenates the projected features of the selected metapaths [8] and does not include any neighbor attention or semantic attention in the final target network. GraphMSE firstly learn the metapath instances with an MLP and then sum them to get the metapath embeddings, which are further aggregated with a graph attention layer of which the weights are regarded as the metapath importance [6].

Figure 3.1: The shown architecture of the Original SeHGNN model.

3.3 SeHGNN as the Backbone Model

Based on the findings discussed above, the model of SeHGNN [12] is designed as the metapath model. It is composed of three elements: simplified neighbor aggregation, multi-layer feature projection and semantic fusion using Transformer [12].

For four reasons, SeHGNN is adopted as the backbone of this thesis. First, it gains competitive accuracy, outperforming the compared baselines on the four datasets of HGB benchmark (except for micro-F1 on Freebase) and outperforming the compared methods on ogbn-mag under the same setting [12]. Second, it trains very quickly: HGB, HAN and MAGNN have an average training time per epoch of 2.3, 25 and 500 times slower than SeHGNN, respectively [12]. Third, it also acts as a reference point for future research: LMSPS use it as their best metapath-based method and re-implement their neighbouragation strategy [8] and PM-HGNN re-implement its metapath enumeration strategy [11]. Most importantly for this thesis, metapaths only come into SeHGNN via the pre-computed feature matrices—one for each metapath in the provided set—and a different set of metapaths can be provided without changing the architecture [12, 8].

The natural addition to the HGNNs is a feature that clearly differentiates SeHGNN from earlier HGNNs based on metapath [12], which is the decoupling of neighbor aggregation from training. However, the method like HAN constructs metadata-path neighbor graphs that have to enumerate all metadata-path neighbors and then aggregates them in each epoch, which is very costly due to the number of metadata-paths grows exponentially according to the length of the metadata-path. However, SeHGNN calculates metapath features once before training.

3.4 Metapath Search and Selection

Manual design of metapaths demands effort and domain knowledge, and may fail to capture useful semantics [13, 6, 2] — thus, several approaches automate the metapath selection. These methods vary as to how and when the selection is made. MetaSieve categorizes meta-path importance learning approaches into two groups: those that learn and design metapath importance together with representation learning within the model (exemplified by [9]); and those that address metapath selection as an external search/optimization problem

(exemplified by manual design, implicit learning by attention, genetic search and differentiable search [3]). With these ideas, the following three categories can be found: (i) implicit or soft selection inside the model during training, (ii) explicit search stages which are coupled to the model training and which are then followed by retraining of the model, and (iii) selection as a preprocessing step prior to training of the downstream model.

The first one is where metapath importance is learnt as part of end to end training. HAN's semantic-level attention was proposed to solve the problem of metapath selection and semantic fusion, but the set of metapaths is provided in advance with weights assigned [7, 9]. GTN eliminates all the metapaths that were manually specified and learns the soft compositions of a variety of edge types between stacked layers [9, 6]; However, every soft composition of edges in the metapath could be obtained by multiplying the corresponding matrices which is equivalent to examining all the metapath instances starting from all nodes, so there is high computational cost [6] and huge memory cost has been reported [3]. GraphMSE samples metapath instances rather than enumerating, aligns the metapath embeddings in a common feature space, and utilizes attention weights as metapath importance [6]. In their experiments, retaining only the top 2–3 metapaths after training led to a slight decline in the accuracy on ACM, and either a slight drop or improvement on DBLP and IMDB points, respectively [6], while LMSPS reported that $LMSP_{350}$'s discovered metapaths had not been as effective as the full set of metapaths [8]. In general, the metapaths implicitly learned by attention are seldom utilized to create the outcomes [8] softmax attention always has a positive value for the noise [8] and the longest metapath need to be still explored as a hyperparameter [6].

The second group turns metapath discovery into explicit search problem, usually based on the neural architecture search (NAS) methods. GEMS' idea is to conduct an evolutionary search for meta graphs for recommendation, and each newly generated meta graph must be re-trained the GNN from scratch, losing the efficiency. DiffMG can enable a graph search in a differentiable manner: graph architecture parameters weight the candidate edge types in a DAG-shaped search space, a task-dependent type constraint eliminates edge types whose message passing will not affect nodes related to the task, and architecture parameters are optimized using a bi-level optimization of training and validation losses [3]. DiffMG is trained in two stages, the search part and the evaluation part where the model is re-trained from scratch based on the results from the search; the search cost is reported to be comparable to the evaluation cost [3]. The number of intermediate states in the search space is predetermined and this limits the number of message-passing steps in the searched structure [3].

LMSPS aims to focus on long-range dependencies over large, heterogeneous graphs and is directly based on the neighbor aggregation that was precomputed by SeHGNN [8]. It scans all the metapaths of the node type within its search space, whose maximum hop is set beforehand. A super-network is designed assigning an architecture parameter to each metapath, and the search space is further reduced by sampling phase after phase (with sampled metapath sets evaluated 200 times each and the set with lowest validation loss chosen), and only the selected metapaths are used to train another target network [8]. However, LMSPS makes clear that the metapaths can be noisy and/or just redundant from one another, so the top-M metapaths by individually estimated importance may not be the best [8]. But the number of selected metapaths is always set as M = 30 for all datasets, maximum hop is set for each dataset in advance, and the authors claim that the best maximum hop has to be specified for each dataset and each task, i.e., it is not automatically determined [8].

AMP-HGNAS also begins with every metapath leading to the target node, and specifically deals with the issue of redundancies [1]. It quantifies the dependency between two metapaths based on the nodes and edges they cover, groups highly-dependent metapaths into a dependency graph, greedily selects a set of non-adjacent metapaths (i.e., nodes that don't overlap) and iteratively optimizes metapath strengths and network weights by alternating updates and subsequently searches over aggregation strategies differentiably [1]. MP-GNN adopts an incremental strategy, consisting of scoring the candidate relations extending the already generated metapath prefix, and greedily generating metapaths based on the relational information gain principle [10]. Its successor MPS-GNN alleviates this "existential" assumption but has to repeatedly train a GNN on the selected metapaths so far, so the search is expensive [9].

The third group choose metapaths prior to training the downstream model. MetaSieve for relational deep learning on multi-table databases (MTD) computes statistics of each candidate metapath extension through SQL queries, estimates the mutual information (MI) between the statistics and the task labels, penalizes the extension that has high sampling cost, considers the coverage and separates good from poor extensions using a two-component Gaussian mixture model [9]. Its selection is based on database statistics and task labels, without depending on the GNN parameters [9]. The maximum hop count is one of the inputs of the method and each extension is evaluated by the mutual information calculated between the statistics of the extension and the label. PM-HGNN also works in a pre-processing phase: based on all metapaths up to a maximum hop, as in SeHGNN, it calculates the ratio of overlaps between metapaths' adjacency matrices with the same

end node type and then merges highly overlapping metapaths based on two thresholds [11]. ASIAM-HGNN filters out the top k metapath instances for each node based on the high frequency appearance in random walks and does not need to first select metapath types [2]. Redundancy of PM-HGNN and ASIAM-HGNN, and frequency of AMP-HGNAS are calculated based on its graph structure (overlap, cover or occurrence frequency), but not based on the task label [11, 1, 2].

When metapaths are selected by each method is summarized in Table 3.2. Most of the works use automatic methods, in which the selection is combined with training of neural models (attention-based methods), or in which the training of a super-network or the training of candidate models is done in a search phase, followed by a training phase of the final model on the selected metapaths (search-based and incremental methods). Only a few of them do selection prior to training and independent of the parameters of the downstream model [9, 11, 2] and only MetaSieve does selection based on the task labels [9].

3.5 Research Gap

The first missing point is how SeHGNN gets the metapaths. Increasing the maximum metapath length will enhance performance by adding more metapaths with different semantics [12] and SeHGNN employs all metapaths up to a certain length (the hop) that are related to the target node type. The number of these metapaths grows exponentially with the maximum hop however [8]. It was observed that the number of metapaths of ogbn-mag is gradually increasing from 10 metapaths (hop 2) to 50 metapaths (hop 4), and 226 metapaths (hop6) (hops of k>4 are not observed due to out of memory problem of SeHGNN) [8]. The semantic-fusion cost of SeHGNN is quadratic with k metapaths [12] and each metapath has a projection MLP structure: for each uninformative metapath, parameters and computation are added to every training step. It is accordingly found that the number of metapaths grown exponentially is a source of the cost of semantic aggregation of SeHGNN [8] and indiscriminate use of all metapaths resulted in excessive computational cost, noise and redundancy [1].

The second one is that of redundancy. The larger the metapath set the more expensive; larger sets can be less effective. However, only the metapath APV is used on DBLP for SeHGNN with maximum hop of 2, which is only slightly inferior to using all of the metapaths, and removing the metapaths PC and PCP on ACM improves Micro-F1 by 0.52% [8]. A long metapath can pass over and over the graph, and therefore may have essentially the

same node connectivity as a shorter metapath; e.g. the overlap rate between PCP & PCPCP is reported as 1 [11]. According to PM-HGNN, the existing approaches do not provide systematic identification and removal of redundant metapaths [11] and AMP-HGNAS observes that SeHGNN and LMSPS do not take all the interdependencies between metapaths into account [1]. However, attention-based fusion can't address the problem because there is performance degradation in SeHGNN showing its attention mechanism is not sufficient to handle noise [8]. The same problem occurs with feature selection, where "the m best features are not the best m features" and individual features can be highly redundant [4]. This means that it is not enough to rank metapaths according to their relevance to the labels; their relevance should be judged regarding already selected metapaths.

The third gap is the each metapath length is also fixed by hand and often the number of selected metapaths. SeHGNN's performance depends on a maximum hop [12, 8] and it has been reported that SeHGNN is sensitive to metapath length and lacks automatic selection of an optimal scale [5]. This limitation is carried over by most automatic selection methods, such as LMSPS, which sets the maximum hop number, or GraphMSE, which searches the maximum hop as a hyperparameter; MP-GNN requires an Lmax to be pre-defined, while DiffMG defines a number of intermediate states, and MetaSieve requires the maximum hop count as an input. GraphMSE also noted that when the search length is too great, selection is more difficult and accuracy suffers [6] — making a large maximum hop by no means the answer!

The fourth one is a trade-off between task-awareness and cost. Structural criteria like adjacency overlap [11] or node and edge coverage [1] or random-walk frequency [2] do not leverage the task labels, but on the other hand, not all semantic information is useful for the downstream task [3]. Task-aware search methods, on the other hand, are developed by training neural models while searching, such as the super-network framework used by bi-level optimization [3, 8] or a new GNN every time the search is extended [10] before training the final model. Although task-aware selection can be carried out as a lightweight pre-processing step by using the mutual information with the labels, as done in MetaSieve [9], it scores each candidate extension by its own statistics, which is designed for sampling subgraphs in relational deep learning instead of selecting metapath set of a pre-computation based HGNN like SeHGNN [9]. We have reviewed several methods in this chapter, but none of them explicitly does (a) reference task labels, (b) considers metapath conditional with regard to the selected ones and hence prevents redundant metapaths, and (c) decides the number and length of the metapaths picked based on data, rather than given a maximum hop.

3.6 Positioning of the Proposed Method

The method proposed in this thesis, fills the above-gap within the SeHGNN framework. It is used as a pre-processing operation prior to SeHGNN without requiring any change in the architecture of SeHGNN since metapaths provided to SeHGNN is a given list of pre-computated inputs [12]. It is about the first gap that it substitutes a subset of a small number k of metapaths that enter per-metapath projections and a quadratic semantic fusion [12] with the use of all metapaths up to a maximum hop. As far as the second gap, each candidate metapath is scored in terms of its conditional information gain about the labels with respect to the metapaths already selected. This is similar to incremental form of Max-Dependency criterion in which the next feature is the one with the highest conditional mutual information to the labels with respect to the selected feature subset [4] (according to the chain rule of mutual information, it is equal to the maximum mutual information to the labels for the selected set). A metapath that is therefore redundant with respect to the selected ones, therefore gets little gain and is not added. For the third gap, the metapaths selected are greedy added to the end until the gain disappears and the number of metapaths selected and their length is consequently determined by the data. The score is defined with respect to the task labels and is computed before training SeHGNN, so the score is a task-aware one in the 4th gap, but it is not coupled with the parameters of SeHGNN. Unlike MP-GNN, which also grows metapaths greedily with an information-gain-inspired score but trains a GNN after every extension [10], and unlike MetaSieve, which also uses label information before training but scores each extension by its own mutual information [9], the proposed method combines conditional scoring with a data-driven stopping criterion for a pre-computation-based HGNN. The method does not change the neighbor aggregation or the semantic fusion of SeHGNN and does not search for meta graphs [3]; its goal is to achieve accuracy comparable to SeHGNN with a considerably smaller set of metapaths.

Table 3.1: Node-level aggregation and semantic-level fusion in representative HGNNs.

Model	Family	Node-level aggregation (within a metapath/relation)	Semantic-level fusion (across metapaths/relations)
HAN [7]	metapath-based	node-level attention	semantic-level attention, weighted sum
MAGNN [13, 12]	metapath-based	attention over metapath instances	inter-metapath aggregation
HetSANN, HGT [13, 12]	metapath-free	type-aware attention over neighbors	not separated from neighbor attention
HGB [12]	metapath-free	GAT attention with learnable edge-type embeddings	not separated from neighbor attention
GraphMSE [6]	metapath-based†	sum of MLP-encoded sampled metapath instances	graph attention over metapath embeddings
SeHGNN [12]	metapath-based	mean aggregation, pre-computed once	Transformer-based semantic fusion
PM-HGNN [11]	metapath-based	similarity-enhanced mean aggregation, pre-computed	weights based on metapath length and type
LMSPS (target net) [8]	metapath-based	mean aggregation as in SeHGNN, pre-computed	no attention; concatenation and MLP

†Classification by the author of this thesis.

Table 3.2: Automatic metapath selection methods: selection signal and the stage at which selection takes place.

Method	Selected unit	Selection signal	Stage	Fixed in advance
HAN [7]	weights of given metapaths	semantic-level attention	during training	metapath set
GraphMSE [6]	top-k metapaths	attention weights	during training	max. length ($l \leq 5$ searched)
DiffMG [3]	meta graph	architecture parameters, bi-level optimization	search stage, then retraining	number of intermediate states K
LMSPS [8]	set of M metapaths	architecture parameters, sampling evaluation	search stage, then retraining	max. hop; $M = 30$
AMP-HGNAS [1]	non-redundant metapath combination	node/edge-coverage dependency; architecture parameters	search with alternating updates	threshold δ
MP-GNN [10]	metapaths grown relation by relation	information-gain-inspired score; validation F1	interleaved with training	$L_{\max}$
MetaSieve [9]	metapath extensions	MI with labels, cost, coverage	pre-processing	max. hop H
PM-HGNN [11]	merged metapath set	adjacency overlap ratio	pre-processing	max. hop; thresholds a, b
ASIAM-HGNN [2]	k instances per node	random-walk frequency	before aggregation	k

Chapter 4

Methodology

This chapter describes the proposed method, **IG-SeHGNN** (Task-aware, Automatic-Depth MetaPath Selection). IG-SeHGNN is a selection step that is placed in front of the Simple and Efficient Heterogeneous Graph Neural Network (SeHGNN). It decides which metapaths are passed to SeHGNN, while SeHGNN itself is used without any modification.

Section 4.1 gives an overview of the method, and Section 4.1 introduces the notation and summarises how SeHGNN uses metapaths. Section 4.3 describes the candidate metapaths, Section 4.4 defines the selection criterion, and Section 4.5 explains how it is estimated in practice. Section 4.6 presents the greedy selection procedure, and Section 4.7 its integration into SeHGNN and its effect on the computational cost. Section 4.8 relates the method to SeHGNN and MP-GNN, Sections **??** discuss limitations and implementation details, and Section 4.10 summarises the chapter. Derivations and further details are collected in Appendix 9.

4.1 Overview

The time complexity of SeHGNN's semantic fusion, however, contains a term that grows quadratically with the number of metapaths [12, Table 5]. A larger hop budget therefore has two effects at once: potentially useful long metapaths are included, but so are metapaths that are uninformative or redundant for the task, and every one of them adds computation. The question addressed in this thesis is therefore *which* metapaths should be passed to the model.

IG-SeHGNN combines these two lines of work. It keeps the SeHGNN architecture and adds a greedy, information-gain-based selection of metapaths that is adapted to the pre-computed metapath channels of SeHGNN. The

method consists of three stages (Figure 4.1):

1. **Candidate generation.** All feature and label metapath channels up to a hop budget are pre-computed with the neighbour aggregation and label propagation of SeHGNN (Section 4.3).

2. **Task-aware selection (proposed).** A small subset of the feature channels is selected by greedily maximising an estimate of the conditional information gain that each channel brings about the task labels (Sections 4.4–4.6).

3. **Training.** The unchanged SeHGNN is trained on the selected feature channels together with all label channels (Section 4.7).

Only the second stage is new. The architecture and all training hyperparameters of SeHGNN are kept identical, so that any difference between SeHGNN and IG-SeHGNN can be attributed to the selected set of metapaths alone. The method follows five design principles:

- **Task-aware:** a metapath is judged by how much it helps to predict the labels of the task, not by the structure of the graph alone.
- **Redundancy-aware:** a metapath is judged *given* the metapaths that are already selected, so that metapaths carrying the same information are not selected twice.
- **Automatic depth:** the depth (number of hops) of the selected metapaths is not fixed in advance; it results from the selection itself.
- **Architecture-preserving:** SeHGNN is not modified; the selection only filters its input channels.
- **No test leakage:** the selection uses only the labels of training and validation nodes; test labels are never accessed.

4.2 Metapath channels

Feature channels. For every target node and every metapath $\mathcal{P} \in \Phi_X$, SeHGNN aggregates the raw features of all metapath-based neighbours with the mean aggregator [12]. Instead of enumerating the neighbours of each metapath, it computes the aggregates of all target nodes at once by multiplying row-normalised adjacency matrices [12]:

$$X^{\mathcal{P}} = \hat{A}_{c,c_1} \hat{A}_{c_1,c_2} \cdots \hat{A}_{c_{l-1},c_l} X^{c_l}, \qquad \mathcal{P} = c\, c_1 c_2 \ldots c_l, \tag{4.1}$$

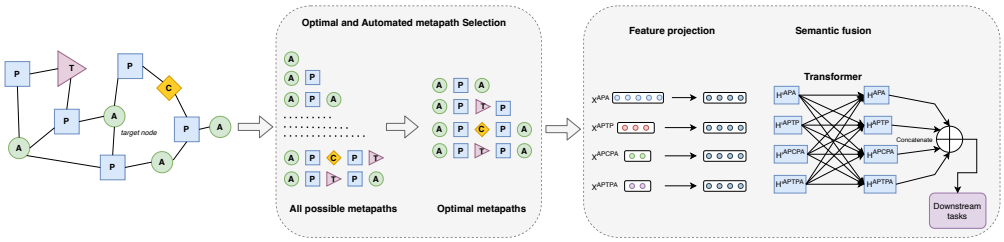


Figure 4.1: Overview of IG-SeHGNN. Left: a heterogeneous graph with a target node. Middle: task-aware automatic metapath selection; all candidate metapaths up to the hop budget are reduced to a small set of selected metapaths. Right: the unchanged SeHGNN backbone (Section 4.2); feature projection of each selected channel and transformer-based semantic fusion, followed by concatenation and the downstream task.

where $\mathcal{P}$ is an l-hop metapath, $\hat{A}_{c_i,c_{i+1}}$ is the row-normalised adjacency matrix between node types c_i and c_{i+1}, and X^{c_l} is the raw feature matrix of type c_l [12]. The results of short metapaths serve as intermediate values for longer ones, e.g. $X^{PPAP} = \hat{A}_{PP}X^{PAP}$ on ACM [12]. SeHGNN also uses the 0-hop metapath, i.e. the target node itself [12, Figure 2]. Because this aggregation has no trainable parameters and involves only linear operations, it is executed only once, in the pre-processing step [12]. Each row of $X^{\mathcal{P}}$ is therefore a fixed vector that describes a target node as seen through metapath $\mathcal{P}$.

Label channels. SeHGNN also represents the training labels in one-hot format and propagates them along metapaths whose two endpoints are of the target type [12]:

$$Y^{\mathcal{P}} = \mathrm{rm_diag}(\hat{A}^{\mathcal{P}})\, Y^{c}, \qquad \hat{A}^{\mathcal{P}} = \hat{A}_{c,c_1}\hat{A}_{c_1,c_2}\cdots\hat{A}_{c_{l-1},c}, \tag{4.2}$$

where the rows of Y^c that correspond to training nodes contain one-hot labels and all other rows are zero [12]. Removing the diagonal prevents each node from receiving its own ground-truth label and thus avoids label leakage [12].

Channel collection. All channels are collected into one set of semantic matrices [12]:

$$\mathcal{M} = \{X^{\mathcal{P}} : \mathcal{P} \in \Phi_X\} \cup \{Y^{\mathcal{P}} : \mathcal{P} \in \Phi_Y\}. \tag{4.3}$$

We call $X^{\mathcal{P}}$ a *feature channel* and $Y^{\mathcal{P}}$ a *label channel*, write Φ for the list of all channels (feature and label) and $k = |\mathcal{M}| = |\Phi|$ for their number.

The SeHGNN backbone

Feature projection. Semantic vectors of different metapaths may have different dimensions or lie in different data spaces. SeHGNN therefore projects each channel with its own MLP, which contains a normalisation layer, a non-linear layer and a dropout layer between consecutive linear layers [12]:

$$H'^{\mathcal{P}} = \mathrm{MLP}_{\mathcal{P}}(\mathcal{M}^{\mathcal{P}}), \qquad \mathcal{P} \in \Phi, \tag{4.4}$$

where $\mathcal{M}^{\mathcal{P}} \in \mathcal{M}$ is the channel of $\mathcal{P}$.

Transformer-based semantic fusion. The projected vectors $\mathbf{h}'^{\mathcal{P}}$ of a node are fused by a transformer-based module [12]. Each vector is mapped to a query $\mathbf{q}^{\mathcal{P}_i} = W_Q \mathbf{h}'^{\mathcal{P}_i}$, a key $\mathbf{k}^{\mathcal{P}_i} = W_K \mathbf{h}'^{\mathcal{P}_i}$ and a value $\mathbf{v}^{\mathcal{P}_i} = W_V \mathbf{h}'^{\mathcal{P}_i}$, and the output of each semantic is an attention-weighted sum of all values plus a residual connection [12]:

$$\alpha_{(\mathcal{P}_i,\mathcal{P}_j)} = \frac{\exp\left(\mathbf{q}^{\mathcal{P}_i} \cdot \mathbf{k}^{\mathcal{P}_j T}\right)}{\sum_{\mathcal{P}_t \in \Phi} \exp\left(\mathbf{q}^{\mathcal{P}_i} \cdot \mathbf{k}^{\mathcal{P}_t T}\right)}, \tag{4.5}$$

$$\mathbf{h}^{\mathcal{P}_i} = \beta \sum_{\mathcal{P}_j \in \Phi} \alpha_{(\mathcal{P}_i,\mathcal{P}_j)}\, \mathbf{v}^{\mathcal{P}_j} + \mathbf{h}'^{\mathcal{P}_i}, \tag{4.6}$$

where W_Q, W_K, W_V and β are trainable parameters shared across all metapaths [12].

Prediction. The final embedding is the concatenation of all output vectors, and another MLP produces the prediction [12]:

$$\mathrm{Pred} = \mathrm{MLP}([\mathbf{h}^{\mathcal{P}_1} \,\|\, \mathbf{h}^{\mathcal{P}_2} \,\|\cdots\|\, \mathbf{h}^{\mathcal{P}_k}]). \tag{4.7}$$

The network is trained with the cross-entropy loss $\mathcal{L} = -\sum_i y_i \ln \mathrm{pred}_i$ [12]. Two findings of SeHGNN motivate this design: semantic attention is essential while neighbour attention is not necessary, and models with a single-layer structure and long metapaths outperform those with multiple layers and short metapaths [12]. Details of the public implementation that matter for the parameter count are summarised in Appendix 9.6.

Why the number of channels matters. Every channel has its own input and projection weights and contributes one d-dimensional slice to the concatenated vector of Eq. (4.7) (Appendix 9.6), and it adds one row and one column to the $k \times k$ matrix of attention weights of Eq. (4.5). In SeHGNN's own analysis of a single layer with k metapaths, n target nodes

and dimension d, feature projection costs $\mathcal{O}(nkd^2)$, semantic fusion costs $\mathcal{O}(n(kd^2 + k^2d))$, and neighbour aggregation causes no cost during training, which gives $\mathcal{O}(nd(k^2 + kd))$ in total [12, Table 5]. SeHGNN uses tens of metapaths at most on its datasets, whereas the neighbour aggregation of HAN and HGB involves thousands of neighbours per node for three or more hops; its theoretical complexity is therefore much lower than theirs [12]. For a given dataset, both the number of parameters and the training time of SeHGNN grow with k.

4.3 Candidate generation

IG-SeHGNN first generates the same candidates that SeHGNN would use (Section 4.2). Given a hop budget L, the candidate set of feature metapaths contains every metapath that starts at the target type and has at most L hops:

$$\Phi_X(L) = \{\mathcal{P} = c\, c_1 \dots c_l \ : \ 0 \le l \le L, \text{ each pair } (c_i, c_{i+1}) \text{ is connected by a relation}\}. \quad (4.8)$$

The 0-hop metapath is the target type itself, i.e. the node's own features. The candidate set $\Phi_Y(L_Y)$ of label metapaths contains the closed metapaths, whose two endpoints are of the target type, with at most L_Y hops. All candidate channels are computed with Eqs. (4.1) and (4.2)

This step applies only **structural** constraints: the length of the metapath, the schema of the graph and, for label channels, the requirement that the metapath is closed. Whether a metapath is useful for the task is not considered at this stage. With $L = L_Y = 4$, the candidate pools of the three HGB datasets used in this thesis are shown in Table 4.1. The pool grows quickly with the number of hops; on ACM, more than half of all feature metapaths are 4-hop metapaths.

Which channels are selected. IG-SeHGNN applies the selection only to the feature channels Φ_X; the label channels Φ_Y are always passed to the model unchanged. There are three reasons for this choice. First, the combinatorial growth takes place on the feature side, because feature metapaths may end at any node type, whereas label metapaths must be closed. Second, label channels directly contain propagated training labels and are among the most informative inputs of the model, so removing them is risky. Third, label

Table 4.1: Size of the candidate pools for $L = L_Y = 4$: number of feature metapaths per hop, total number of feature metapaths $|\Phi_X|$, number of label metapaths $|\Phi_Y|$ and total number of channels k.

Dataset (target)	0-hop	1-hop	2-hop	3-hop	4-hop	$\lvert\Phi_X\rvert$	$\lvert\Phi_Y\rvert$	k
DBLP (author)	1	1	3	3	9	17	4	21
ACM (paper)	1	3	5	11	21	41	20	61
IMDB (movie)	1	3	3	9	9	25	12	37

channels are built from the labels of the training nodes (Eq. (4.2)); scoring them with a cross-validation procedure that also uses these labels would require an additional, fold-specific recomputation to avoid leakage. The target node's own feature channel is always kept, because the model uses it as an input and in its residual connection.

4.4 Selection criterion: conditional information gain

Throughout this section, Y denotes the label of a randomly drawn labelled node and $X^{\mathcal{P}}$ its vector in channel $\mathcal{P}$. Natural logarithms are used, so all quantities are measured in nats. The entropy $H(Y)$ measures the uncertainty about the label, the conditional entropy $H(Y \mid X)$ measures the uncertainty that remains after X has been observed, and the mutual information $I(X;Y) = H(Y) - H(Y \mid X)$ is the reduction of uncertainty obtained by observing X. In classification, $I(X;Y)$ is the classical information gain of X with respect to the label. The formal definitions and their basic properties are given in Appendix 9.1.

For metapath selection, the relevant question is not how informative a channel is on its own, but how much it adds to the channels that are already selected. Let $X^S = \{X^{\mathcal{Q}} : \mathcal{Q} \in S\}$ denote the channels of a selected set S. The **conditional mutual information** between a candidate channel $X^{\mathcal{P}}$ and the label, given X^S,

$$I(X^{\mathcal{P}}; Y \mid X^S) = H(Y \mid X^S) - H(Y \mid X^S, X^{\mathcal{P}}), \tag{4.9}$$

is the uncertainty about the label that is still present after observing X^S and that disappears when $X^{\mathcal{P}}$ is observed as well. By the chain rule, the information carried by channels that are added one after the other decomposes into

such conditional terms:

$$I(X^{\mathcal{P}_1}, \ldots, X^{\mathcal{P}_m}; Y) = \sum_{j=1}^{m} I(X^{\mathcal{P}_j}; Y \mid X^{\mathcal{P}_1}, \ldots, X^{\mathcal{P}_{j-1}}). \tag{4.10}$$

Equation (4.9) makes two situations precise that are central to metapath selection:

- **Redundancy.** If $X^{\mathcal{P}}$ carries only information that X^S already contains, then $I(X^{\mathcal{P}}; Y \mid X^S) = 0$, even if $X^{\mathcal{P}}$ is highly informative on its own ($I(X^{\mathcal{P}}; Y) > 0$).
- **Complementarity.** A channel that is weak on its own can be very informative in combination with others; $I(X^{\mathcal{P}}; Y \mid X^S)$ can be larger than $I(X^{\mathcal{P}}; Y)$.

The ideal value of adding metapath $\mathcal{P}$ to the selected set S is therefore its conditional information gain

$$\mathrm{IG}(\mathcal{P} \mid S) = I(X^{\mathcal{P}}; Y \mid X^S). \tag{4.11}$$

By Eq. (4.10), adding in every step the metapath with the largest IG increases the information about the labels that is captured by the selected set as much as possible in that step. Appendix 9.2 gives a small worked example in which ranking the channels by their individual information gain selects a redundant copy, whereas the conditional criterion selects a complementary channel that is useless on its own.

4.5 Estimating the information gain with a probe

The quantities of Section 4.4 require the true conditional distribution $p(y \mid x)$. For real metapath channels, this distribution is unknown, and the channels are high-dimensional, continuous vectors (several thousand dimensions on IMDB), so the entropies cannot be computed by counting. IG-SeHGNN therefore estimates them with a small classifier, called a **probe**, whose prediction loss on held-out nodes approximates the conditional entropy.

The bag-of-channels probe

For a set of channels S, the probe gives every channel its own linear map and sums the results into class scores:

$$q_S(y \mid \mathbf{x}_i) = \mathrm{softmax}\Big(\sum_{\mathcal{Q} \in S} W_{\mathcal{Q}}\, \mathbf{x}_i^{\mathcal{Q}}\Big)_y, \qquad W_{\mathcal{Q}} \in \mathbb{R}^{C \times d_{\mathcal{Q}}}. \tag{4.12}$$

Each channel has its own map because channels have different dimensions and lie in different spaces, the same reason for which SeHGNN projects every metapath with its own MLP (Section 4.2). Channels are therefore never compared with each other directly. The probe is trained on a set of training nodes by minimising the cross-entropy with a small weight decay λ. As an optional, more expressive variant, the channel maps project into a hidden space of size h, and a shared non-linear head produces the logits.

Cross-entropy and conditional entropy. The expected cross-entropy of a probe under the true data distribution p decomposes as

$$\mathrm{CE}(q_S) = \mathbb{E}_{(x,y)\sim p}[-\ln q_S(y \mid x)] = H(Y \mid X^S) + \Delta(S), \tag{4.13}$$

where $\Delta(S) = \mathbb{E}_x[\mathrm{KL}(p(\cdot \mid x) \,\|\, q_S(\cdot \mid x))] \geq 0$ (derivation in Appendix 9.3). Because the Kullback–Leibler divergence is non-negative, the cross-entropy of any probe is an upper bound of the conditional entropy, with equality exactly when the probe predicts the true conditional distribution. The term $\Delta(S)$ is the *approximation gap* of the probe.

Held-out score

The cross-entropy is estimated on nodes that were **not** used to train the probe. The probe for $S \cup \{\mathcal{P}\}$ contains the probe for S as a special case (set $W_{\mathcal{P}} = 0$), so its training loss can never be higher. Measured on the training nodes, every additional channel would appear useful, even a channel that carries no information and only allows the probe to over-fit. Measured on held-out nodes, a channel lowers the loss only if it generalises.

To make the estimate stable and independent of a single split, the labelled nodes V_L are divided into F folds $V_1, \ldots, V_F$. For fold t, the probe $q_S^{(t)}$ is trained on $V_L \setminus V_t$ and evaluated on V_t. The **score** of a set of channels is the negative average held-out cross-entropy, so that higher is better:

$$\mathrm{Score}(S) = -\frac{1}{F}\sum_{t=1}^{F} \mathrm{CE}_{V_t}(q_S^{(t)}), \qquad \mathrm{CE}_{V_t}(q_S^{(t)}) = -\frac{1}{|V_t|}\sum_{i\in V_t} \ln q_S^{(t)}(y_i \mid \mathbf{x}_i), \tag{4.14}$$

so that $\mathrm{Score}(S) \approx -H(Y \mid X^S)$. The fold partition is fixed; the score therefore depends only on the training and validation labels and not on the random seed that is later used to train SeHGNN. Without any channel, the probe can only predict the class frequencies of the training part, and $\mathrm{Score}(\emptyset) \approx -H(Y)$ is the starting point of the selection.

The estimated information gain

The estimated gain of adding $\mathcal{P}$ to S is the increase of the score, i.e. the decrease of the held-out cross-entropy:

$$\widehat{\mathrm{IG}}(\mathcal{P} \mid S) = \mathrm{Score}(S \cup \{\mathcal{P}\}) - \mathrm{Score}(S) \;\approx\; I(X^{\mathcal{P}}; Y \mid X^{S}). \tag{4.15}$$

The approximation error consists of the change of the approximation gap when $\mathcal{P}$ is added and the sampling error of the held-out estimate (Appendix 9.3). Two consequences follow. First, the true conditional mutual information is never negative, but the estimate can be: a negative $\widehat{\mathrm{IG}}$ indicates that the channel lets the probe over-fit, which happens in particular for very high-dimensional channels with few training nodes. Second, the estimated gains of a sequence of selected channels sum to $\mathrm{Score}(S_m) - \mathrm{Score}(\emptyset)$, which estimates $I(X^{S_m}; Y)$, the total information about the labels captured by the selected set; this quantity is bounded by $H(Y)$.

Multi-label tasks. For the multi-label dataset IMDB, the probe outputs one logit per class, models each label independently with a sigmoid, and is scored with the binary cross-entropy averaged over the classes. The argument above applies unchanged, and the gain estimates the conditional mutual information in nats per class (Appendix 9.4).

What makes the criterion task-aware. The score of Eq. (4.14) is computed from the labels of the task. The same metapath can therefore receive a high gain for one task and a low gain for another, and two metapaths are considered redundant when they carry the *same information about the labels*, not when they share edges or look similar. A purely structural criterion, by contrast, selects the same metapaths for every task on the same graph. Node features enter the criterion through the mean aggregation of Eq. (4.1).

4.6 Greedy selection with automatic depth

Forward selection

The goal is a small set $S \subseteq \Phi_X$ with a high score. Evaluating all subsets is infeasible, so IG-SeHGNN builds the set greedily, one metapath per step, starting from $S_0 = \emptyset$:

$$\mathcal{P}_j = \operatorname*{arg\,max}_{\mathcal{P} \in \Phi_X \setminus S_{j-1}} \widehat{\mathrm{IG}}(\mathcal{P} \mid S_{j-1}) = \operatorname*{arg\,max}_{\mathcal{P} \in \Phi_X \setminus S_{j-1}} \mathrm{Score}(S_{j-1} \cup \{\mathcal{P}\}), \qquad S_j = S_{j-1} \cup \{\mathcal{P}_j\}. \tag{4.16}$$

The two forms are equivalent because $\mathrm{Score}(S_{j-1})$ does not depend on $\mathcal{P}$. The complete procedure is given as GreedySelect in the lower part of Algorithm 1.

Stopping rule and output

The procedure keeps the best score s^* reached so far, the prefix S^* at which it was reached, and a counter of consecutive steps without improvement. A step counts as an improvement only if it raises the best score by more than the threshold ε (in nats); then s^* and S^* are updated and the counter is reset, otherwise the counter is increased. The selection stops when there has been no improvement for π consecutive steps and at least m metapaths have been selected, when M metapaths have been selected, or when no candidate is left. The patience π prevents the procedure from stopping at a single noisy step, and M bounds the cost of the search.

The procedure returns the best prefix, not the last visited set. The metapaths added after it, which were only evaluated to confirm the plateau, are discarded; if the best prefix contains fewer than m metapaths, the first m selected metapaths are used instead. Finally, the target node's own channel c is added if it was not selected:

$$\Phi_X^* = S^* \cup \{c\}. \tag{4.17}$$

Automatic depth

The hop budget L only defines the pool of candidates. The depth that is actually used, $D(\Phi_X^*) = \max_{\mathcal{P}\in\Phi_X^*} \mathrm{hops}(\mathcal{P})$, and the mixture of depths among the selected metapaths are results of the selection. A long metapath is selected only if it adds information that the shorter ones do not already provide, and the selection stops when no metapath of any length adds more than ε. Unlike the default configuration of SeHGNN, in which all metapaths up to the maximum hop are used (Section 4.1), IG-SeHGNN can therefore work with a generous budget L and still keep only the useful long metapaths. The selected set may, for example, contain the 0-hop channel, a 1-hop metapath and several 4-hop metapaths, while skipping all 2-hop metapaths.

Properties and limits of the greedy procedure

The procedure is standard forward-stepwise selection and does **not** guarantee an optimal subset. Because each step conditions on the selected set, it handles redundancy and the type of complementarity shown in Appendix 9.2,

where a channel that is useless alone becomes informative after another channel has been selected. It can, however, miss a group of channels that are informative only jointly, if none of them is informative in combination with the already selected set alone. The search visits at most M steps and evaluates every remaining candidate at each step.

4.7 Integration with SeHGNN and computational effect

Overall procedure

Algorithm 1 shows the complete procedure. Lines 1–3 and 7–16 of its upper part follow the overall training process of SeHGNN (Algorithm 1 in [12]). Lines 4–6, marked by the vertical bar, are the only addition, and the procedure GreedySelect in the lower part implements Section 4.6.

Effect on the number of parameters

Selection reduces the number of parameters in two ways. It lowers the number of channels k, which removes per-channel projection weights and the corresponding part of the layer after the concatenation, and it removes the input matrices of the discarded feature channels, which are large when the raw features are high-dimensional. An approximate parameter count and a worked example for IMDB are given in Appendix 9.5.

Effect on the training time

With the complexity of Section 4.2, IG-SeHGNN reduces the number of channels from $k = |\Phi_X|+|\Phi_Y|$ to $k' = |\Phi_X^*|+|\Phi_Y|$, which gives a per-epoch cost of

$$\mathcal{O}(nd\,(k'^2 + k'd)), \qquad k' < k. \tag{4.18}$$

The attention term shrinks by the factor $(k/k')^2$ and the projection term by k/k'. For example, a reduction from $k = 37$ to $k' = 15$ channels reduces the attention term by a factor of about 6.1. Because the projection term is only linear in k, the measured speed-up per epoch is smaller than $(k/k')^2$.

Cost of the selection itself

The selection is executed once, before training, and its cost does not depend on the number of training epochs. It fits F probes for the empty set and

Algorithm 1 The overall training process of IG-SeHGNN

Input: raw features $\{X^{c_i}\}$, labels Y, hop budgets L and L_Y, feature-metapath candidates Φ_X, label metapaths Φ_Y

Params: F folds, min/max selected metapaths (m, M), threshold ε, patience π, probe q

Output: predictions Pred for target type c

▷ *Neighbour aggregation (pre-processing, once)*
for $\mathcal{P} \in \Phi_X$**:** $X^{\mathcal{P}} = \hat{A}_{c,c_1} \cdots \hat{A}_{c_{l-1},c_l} X^{c_l}$ *(mean aggr.)*
for $\mathcal{P} \in \Phi_Y$**:** $Y^{\mathcal{P}} = \mathrm{rm_diag}(\hat{A}_{c,c_1} \cdots \hat{A}_{c_{l-1},c})\, Y$ *(label prop.)*
▷ *Task-aware automatic metapath selection (ours)*
$\Phi_X^* \leftarrow \textsc{GreedySelect}(\Phi_X, \{X^{\mathcal{P}}\}, Y_{V_L};\, F, m, M, \varepsilon, \pi)$
$\mathcal{M} \leftarrow \{X^{\mathcal{P}} : \mathcal{P} \in \Phi_X^*\} \cup \{Y^{\mathcal{P}} : \mathcal{P} \in \Phi_Y\}$ *(labels unchanged)*
▷ *Unchanged SeHGNN backbone* **for** *each epoch* **do**
end
$H'^{\mathcal{P}} = \mathrm{MLP}_{\mathcal{P}}(\mathcal{M}^{\mathcal{P}}), \quad \mathcal{M}^{\mathcal{P}} \in \mathcal{M}$ *(feature projection)*
$Q^{\mathcal{P}} = W_Q H'^{\mathcal{P}}, \quad K^{\mathcal{P}} = W_K H'^{\mathcal{P}}, \quad V^{\mathcal{P}} = W_V H'^{\mathcal{P}}$
$\alpha_{ij} = \mathrm{softmax}_j(Q^{\mathcal{P}_i} \cdot {K^{\mathcal{P}_j}}^T)$ *(semantic fusion)*
$H^{\mathcal{P}_i} = \beta \sum_j \alpha_{ij} V^{\mathcal{P}_j} + H'^{\mathcal{P}_i}$
$H^c = \mathrm{concat}([H^{\mathcal{P}_1} \| H^{\mathcal{P}_2} \| \cdots])$
$\mathrm{Pred} = \mathrm{MLP}(H^c); \quad \mathrm{loss} = -\sum_i y_i \ln \mathrm{pred}_i; \quad$ update

return Pred

Procedure $\textsc{GreedySelect}(\Phi_X, \{X^{\mathcal{P}}\}, Y_{V_L};\, F, m, M, \varepsilon, \pi)$

split the training and validation nodes V_L into F fixed CV folds
$S \leftarrow \emptyset; \quad s^* \leftarrow \mathrm{Score}(\emptyset); \quad S^* \leftarrow \emptyset; \quad \mathrm{noimp} \leftarrow 0$
while $|S| < M$ **and** $\Phi_X \setminus S \neq \emptyset$ **do**
  $\mathcal{P} \leftarrow \arg\max_{\mathcal{P}' \in \Phi_X \setminus S} \widehat{\mathrm{IG}}(\mathcal{P}' \mid S)$
  $S \leftarrow S \cup \{\mathcal{P}\}$
  if $\mathrm{Score}(S) > s^* + \varepsilon$**:** $s^* \leftarrow \mathrm{Score}(S); \; S^* \leftarrow S; \; \mathrm{noimp} \leftarrow 0$
  else: $\mathrm{noimp} \leftarrow \mathrm{noimp} + 1$
    if $\mathrm{noimp} \geq \pi$ **and** $|S| \geq m$**: break** *(auto-depth stop)*
if $|S^*| < m$**:** $S^* \leftarrow$ first m selected metapaths *(minimum size)*
return $S^* \cup \{c\}$ *(best-CV prefix + target)*

$\mathrm{Score}(S) = -\frac{1}{F} \sum_{t=1}^{F} \mathrm{CE}_{V_t}(q_S^{(t)})$ Eq. (4.14)

$\widehat{\mathrm{IG}}(\mathcal{P} \mid S) = \mathrm{Score}(S \cup \{\mathcal{P}\}) - \mathrm{Score}(S) \;\approx\; I(X^{\mathcal{P}}; Y \mid X^S)$ Eq. (4.15)

F probes for every remaining candidate in each greedy step. Its cost therefore grows linearly with the number of candidates, the number of folds and the number of probe optimisation steps, and roughly quadratically with the number of greedy steps, because later probes contain more channels (Appendix 9.5). Two further points should be noted. First, the pre-processing step is unchanged: all candidate channels in $\Phi_X(L)$ must be computed before they can be scored, so the savings of IG-SeHGNN concern training, not pre-processing. Second, the selection time must be added to the training time when end-to-end efficiency is reported; this is done in Chapter 7.

4.8 Relation to MP-GNN and SeHGNN

Relation to MP-GNN

MP-GNN learns meta-paths for node classification relation by relation, in a greedy, incremental approach that uses the idea of relational information gain to score candidate extensions of a meta-path prefix [10]. In the first iteration, a candidate relation is scored by how well it explains the node labels: a learnable weight vector accounts for the node features, a learnable weight per neighbour is close to 1 if the neighbour is predicted to belong to the ground-truth meta-path and close to 0 otherwise, and the score is the minimal mean squared error between the predicted and the true labels; the relation with the minimum score is selected [10]. For the next relation, a positive node is replaced by the bag of its neighbours that are not neighbours of any negative node, negative bags are singletons, and a candidate relation is scored by its potential to predict the bag labels [10]. The equations are summarised in Appendix 9.7.

After each extension, an MP-GNN built on the current meta-path prefix is trained, its F1 score on a validation set is computed, and the prefix with the highest F1 is kept [10]. The authors describe this F1 score as the only "real" measure of meta-path quality, whereas the scoring function measures a "potential" informativeness [10]. Multiple meta-paths are handled with a beam search, with a beam size of three in the experiments; meta-paths that do not increase F1 are discarded, and the maximal meta-path length is set to four [10]. The method is designed for existential meta-path features, i.e. cases where the existence of a meta-path instance is informative for the class label [10]. It assumes that the informativeness of a meta-path also depends on the features of the traversed nodes, and its experiments on FB15K-237 show that node features are crucial for leveraging meta-path information [10]. On these knowledge-graph tasks, which have a larger set of candidate relations,

Table 4.2: Comparison of MP-GNN and IG-SeHGNN.

Aspect	MP-GNN [10]	IG-SeHGNN (this thesis)
Unit of selection	one relation, appended to a meta-path prefix	one complete, pre-computed metapath channel
Score	minimal MSE of a pseudo-label model with a maximum over neighbours (Appendix 9.7)	held-out, cross-validated cross-entropy of a bag-of-channels probe (Eq. (4.14))
What the score measures	"potential" informativeness of a relation	estimate of $I(X^{\mathcal{P}}; Y \mid X^{S})$ (Eq. (4.15))
Conditioning on previous choices	through labelled bags along one meta-path; several meta-paths through beam search	explicitly on the set of already selected channels
Model training during the search	an MP-GNN is trained after every extension to obtain the validation F1	none; only small probes are fitted, and SeHGNN is trained once
Output	prefix with the highest validation F1; maximal length four	best cross-validated prefix; stopping rule with ε and π
Downstream model	MP-GNN, a variant of RGCN	the unchanged SeHGNN

its meta-path learning is more expensive than most competitors [10].

IG-SeHGNN adopts three ideas from MP-GNN [10]: meta-paths are selected greedily and incrementally with an information-gain-based score, their informativeness is judged with respect to the task labels and with the node features taken into account, and the best-scoring prefix is returned. It differs in the unit of selection, in the score and in the cost of the search (Table 4.2).

Relation to SeHGNN

IG-SeHGNN keeps every component of SeHGNN: the mean aggregator in pre-processing, the multi-layer feature projection, the transformer-based semantic fusion and the label propagation [12]. In particular, it keeps the transformer-based fusion, whose replacement by the weighted-sum fusion of HAN (Variant#4) lowers the accuracy of SeHGNN, and the single-layer structure with long metapaths that SeHGNN's second finding favours [12]. What IG-SeHGNN changes is the rule that decides which metapaths enter the model: instead of all metapaths up to the maximum hop, only those that

Table 4.3: Hyper-parameters of the selection step.

Parameter	Symbol	Value
Hop budget, feature / label metapaths	L / L_Y	4 / 4
Number of folds (fixed partition, seed 0)	F	5
Scoring function	–	held-out cross-entropy (binary cross-entropy for IMDB)
Probe	–	linear bag-of-channels (optional MLP head with $h = 128$)
Probe optimisation	–	Adam, learning rate 0.05, weight decay 10^{-4}, 100 full-batch steps
Improvement threshold	ε	10^{-3} by default; studied in $\{5 \cdot 10^{-4}, 10^{-3}, 5 \cdot 10^{-3}, 10^{-2}\}$
Patience	π	2
Minimum / maximum number of selected metapaths	m / M	3 by default (1 studied as alternative) / 12

add information about the labels are used. The long metapaths favoured by the second finding can therefore be offered to the model without paying for all of them.

4.9 Implementation details

The method is implemented on top of the public SeHGNN code (Section 4.1). The hyper-parameters of the selection step are listed in Table 4.3. The values of ε and m used for each dataset, and their effect on accuracy and cost, are reported in Chapters 5 and 7. All SeHGNN hyper-parameters are those of the corresponding baseline.

4.10 Summary

IG-SeHGNN adds a single step to SeHGNN: before training, it selects the feature metapaths that carry the most information about the task labels. The criterion is the conditional information gain $I(X^{\mathcal{P}}; Y \mid X^{S})$, i.e. the uncertainty about the label that a metapath removes given the metapaths already selected, which is estimated by the decrease in held-out cross-entropy

of a bag-of-channels probe. Metapaths are added greedily until this gain vanishes, and the best-scoring prefix is kept, so that redundant metapaths are excluded and the depth of the selected metapaths is determined by the data. The greedy, information-gain-based construction follows the idea of MP-GNN, while the score is a direct estimate of conditional mutual information on the pre-computed channels of SeHGNN. Because SeHGNN is left unchanged, the only effect of the method is a smaller set of channels, which reduces the number of parameters and the quadratic semantic-fusion cost of SeHGNN.

Chapter 5

Experimental Setup

This chapter describes the setting in which IG-SeHGNN is evaluated. Section 5.1 introduces the datasets, Section 5.2 the methods with which IG-SeHGNN is compared, and Section 5.3 the evaluation metrics. Section 5.4 lists the hyper-parameters and implementation details, and Section 5.5 gives an overview of the experiments, whose results are reported in Chapters 6 and 7.

5.1 Datasets

The experiments are conducted on three heterogeneous graphs from the Heterogeneous Graph Benchmark (HGB): DBLP, ACM and IMDB. SeHGNN was evaluated on these three datasets and on Freebase from the same benchmark [12], and AMP-HGNAS, LMSPS and PM-HGNN also report results on DBLP, ACM and IMDB [1, 8, 11]. Each dataset contains one target node type, and the task is to predict the labels of the nodes of this type [8]. DBLP is a bibliographic network of computer-science publications from four research areas, with the node types author, paper, term and venue; its target type is author, with four classes [11]. ACM is an academic citation network of papers from three research areas (data mining, database and wireless communication), with the node types author, paper, subject and field; its target type is paper, with three classes [11]. IMDB describes movies with the node types movie, actor, director and keyword; its target type is movie, and movies are categorised into the five genres action, romance, drama, thriller and comedy [11]. IMDB is a multi-label dataset [11], i.e. a movie can belong to more than one genre, whereas DBLP and ACM are single-label datasets. The statistics of the three datasets are summarised in Table 5.1.

All experiments address semi-supervised node classification in the trans-

Table 5.1: Statistics of the datasets as reported in [8, 11]. The numbers refer to the datasets as provided by HGB; the pre-processing of ACM is described in Section 5.1.

Dataset	#Nodes	#Node types	#Edges	#Edge types	Target	#Classes
DBLP	26,128	4	239,566	6	author	4
ACM	10,942	4	547,872	8	paper	3
IMDB	21,420	4	86,642	6	movie	5

ductive setting of HGB, in which all edges are available during training [8]. HGB divides the target nodes into 24% training, 6% validation and 70% test nodes [8]. The public implementation of SeHGNN [12], which is used for all models in this thesis,[1] keeps the test nodes of the benchmark and, in every run, shuffles the remaining labelled nodes with the random seed of the run and divides them into training and validation nodes in the ratio 80:20, which preserves the proportions of 24% and 6%. The selection step of IG-SeHGNN uses the union V_L of training and validation nodes (Section 4.5), which is the same in all runs; the labels of the test nodes are never used.

The graphs are loaded and pre-processed with the unchanged code of this implementation. Two of its pre-processing steps determine the candidate metapaths of ACM in Table 4.1. First, the field nodes are not kept as a separate node type, because the raw features of the papers already represent the distribution of their neighbouring field nodes. Second, the citation and the reference relations between papers [11] are merged into a single paper–paper relation. The feature metapaths of ACM are therefore formed from the remaining three node types.

5.2 Baselines

IG-SeHGNN is compared with three groups of methods, which are summarised in Table 5.2. The first group is SeHGNN itself. Because IG-SeHGNN uses SeHGNN without modification and changes only the set of feature metapaths that it receives (Chapter 4), this comparison isolates the effect of the selection. SeHGNN is run in two configurations. *SeHGNN (paper setting)* uses, for each dataset, the configuration that is published with the official implementation, whose repository is linked in the SeHGNN paper [12]: feature metapaths of at most two hops on DBLP and of at most four hops

[1] Official implementation of SeHGNN [12]: `github.com/ICT-GIMLab/SeHGNN`, directory `hgb`. All file names in the footnotes of this chapter refer to this directory.

on ACM and IMDB, and label metapaths of at most four hops on all three datasets.[2] *SeHGNN (hop-4)* uses a hop budget of four for feature and label metapaths on all three datasets, i.e. it is trained on the complete candidate set of Table 4.1 from which IG-SeHGNN selects. Both models then receive the same candidate feature metapaths and the same label metapaths and differ only in whether the feature metapaths are selected. Since the published configuration already uses a hop budget of four for feature and label metapaths on ACM and IMDB, the two configurations coincide on these datasets and differ only on DBLP, where the number of feature metapaths grows from 5 to 17 (Table 4.1). SeHGNN (paper setting) is trained by us on all three datasets, both to verify that the published results can be reproduced and as the reference for the accuracy and cost comparisons; on DBLP, SeHGNN (hop-4) is trained in addition as the reference with the same candidate set. The results published in [12] are reported as well.

The second group consists of HGNNs whose results are quoted from the literature in order to place IG-SeHGNN in the context of existing methods. From the comparison on the HGB datasets in [12], the results of the four metapath-based methods RGCN, HetGNN, HAN [7] and MAGNN and of the four metapath-free methods RSHN, HetSANN, HGT and HGB are quoted, following the grouping of [12]. HGB denotes the model proposed with the benchmark, which uses a multi-layer GAT network as the backbone and combines node features with learnable edge-type embeddings to compute the attention values [12]; LMSPS reports the same results for this model under the name Simple-HGN [8]. These methods were not re-run in this thesis.

The third group consists of two methods that, like IG-SeHGNN, determine the metapaths automatically and report results on DBLP, ACM and IMDB. LMSPS searches for a fixed number of $M = 30$ metapaths among all metapaths that are related to the target node type and do not exceed a preset maximum hop, which is six on DBLP and five on ACM and IMDB, and it is an MLP-based method [8]. AMP-HGNAS combines a metapath dependency search, which selects informative metapath combinations while minimising redundancy, with an adaptive aggregation search space, and its search follows a differentiable architecture search strategy [1]. The results of both methods are quoted from their papers ([8, Table 1] and [1, Table 3]); neither method was re-run.

Three differences limit the comparability of the quoted results. First, LMSPS reports its own results and those of all baselines in its main comparison without label propagation, and it uses the name SeHGNN for the

[2] `Readme.md`, run commands for DBLP, ACM and IMDB (options `--num-hops` and `--num-label-hops`).

variant of SeHGNN without label propagation [8]. Accordingly, the values it reports for SeHGNN are identical to those of Variant#3 in [12], which does not use labels as extra inputs. In contrast, the SeHGNN results published in [12], as well as all SeHGNN and IG-SeHGNN models of this thesis, use label propagation. Second, AMP-HGNAS is available as a preprint that has not been peer reviewed [1], and the results that it reports for SeHGNN differ from those published in [12]; on DBLP, for example, it reports a Micro-F1 score of 95.06 instead of 95.42 [1, 12]. Third, the quoted results were obtained with different implementations, hardware and hyper-parameter tuning. The quoted results are therefore used only as context, and conclusions about the effect of the selection are drawn from the comparison with SeHGNN under identical conditions.

5.3 Evaluation Metrics

Following the evaluation standard of HGB, classification performance is measured by the Macro-F1 and Micro-F1 scores on the test nodes [8, 12]. Micro-F1 computes a single F1 score from the true positives, false positives and false negatives counted over all classes together, so that frequent classes have a larger influence. Macro-F1 computes the F1 score of every class separately and takes their unweighted mean, so that every class counts equally. Both scores are computed with the evaluation function of the SeHGNN implementation, which uses the `f1_score` function of scikit-learn with micro and macro averaging. On DBLP and ACM, the predicted class of a node is the class with the highest score; on the multi-label dataset IMDB, a genre is predicted whenever its predicted probability exceeds 0.5. The quoted results are the means and standard deviations reported in the respective papers; LMSPS, for example, reports the mean and standard deviation over five runs with different random seeds on the HGB datasets [8]. Efficiency is measured by four quantities. (i) The number of channels k that SeHGNN receives, i.e. the number of feature metapaths plus the number of label metapaths; for IG-SeHGNN, the number $|\Phi_X^*|$ of selected feature metapaths is reported as well. (ii) The number of parameters of the model, counted over all parameter tensors as in the SeHGNN implementation. (iii) The average training time per epoch. SeHGNN and LMSPS compare the efficiency of HGNNs by the average time of one training epoch, and LMSPS reports the number of parameters as well [8, 12]. The SeHGNN implementation measures this time as the wall-clock duration of the training pass over all training batches of an epoch and averages it over all epochs of a run; the evaluation after each epoch is not included. (iv) The wall-clock time of the selection step,

i.e. of the procedure GreedySelect in Algorithm 1. Because the selection is executed only once before training, its time is reported separately from the per-epoch training time, as LMSPS does for the time of its search [8]. The reduction in parameters and the speed-up per epoch of IG-SeHGNN are computed relative to each SeHGNN configuration with which it is compared; on DBLP, these are SeHGNN (paper setting) and SeHGNN (hop-4). The pre-processing step, in which all candidate channels are computed, is the same for SeHGNN (hop-4) and IG-SeHGNN (Section 4.7) and is therefore not part of the comparison.

5.4 Hyper-parameters and Implementation

The SeHGNN backbone of all models, including IG-SeHGNN, uses the hyper-parameters of the published configuration of the respective dataset (Table 5.3). On all three datasets, every channel is embedded into $d_e = 512$ dimensions and projected by a two-layer MLP with hidden dimension $d = 512$, and the dropout rate is 0.5. The number of layers of the downstream MLP, the input dropout and the residual connection from the target node's own features differ between the datasets. All options that the published configuration does not set keep the default values of the implementation: Adam with a learning rate of 10^{-3} and no weight decay, a batch size of 10,000 and no attention dropout. The published configuration also enables mixed-precision training, but the implementation applies it only on a GPU, so it is not active in our runs, which were executed on a CPU. Each model is trained for at most 200 epochs. After every epoch, the validation loss is computed and the model with the lowest validation loss is kept; training stops when the validation loss has not improved for more than 50 epochs. The loss function is the cross-entropy on DBLP and ACM and the binary cross-entropy on IMDB. Every configuration is trained with [five] random seeds, and the tables report the mean test score over these runs; Table 6.2 also reports the standard deviation. SeHGNN (paper setting) and SeHGNN (hop-4) differ only in the hop budget of the feature metapaths on DBLP, and SeHGNN (hop-4) and IG-SeHGNN differ only in the set of feature metapaths.

The selection step uses the same hyper-parameters on all three datasets; they are not tuned per dataset. The main configuration uses the default values of Table 4.3: a hop budget of $L = L_Y = 4$, $F = 5$ cross-validation folds with a fixed partition, a linear bag-of-channels probe trained with Adam (learning rate 0.05, weight decay 10^{-4}, 100 full-batch steps), an improvement threshold of $\varepsilon = 10^{-3}$, a patience of $\pi = 2$, and a minimum and maximum of $m = 3$ and $M = 12$ selected metapaths. The selection is executed once per

dataset before training. Its fold partition does not depend on the training seed, but the probe weights are initialised randomly, and no probe seed was fixed in the main configuration; the ablation studies use a fixed probe seed (Section 7).

All models were trained and timed on the same machine, so that the measured training times are comparable. The implementation of SeHGNN is used without changes; the selection step is implemented in separate files and is switched on by a command-line option.[3]

5.5 Experiments

The experiments are grouped as follows. The first four are reported in Chapter 6 and the last three in Chapter 7.

1. **Reproduction of SeHGNN.** Our runs of SeHGNN (paper setting) are compared with the results published in [12], to verify the implementation and the environment on which the cost comparisons rely (Section 6.1).

2. **Classification accuracy.** IG-SeHGNN in its main configuration is compared with SeHGNN and with the quoted methods of Table 5.2 in terms of Macro-F1 and Micro-F1 (Section 6.2).

3. **Computational cost.** The number of channels, the number of parameters and the training time per epoch of IG-SeHGNN are compared with those of SeHGNN (paper setting) and, on DBLP, of SeHGNN (hop-4) (Section 6.3).

4. **Selected metapaths.** The feature metapaths selected in the main configuration are compared with those used by SeHGNN (Section 6.4).

5. **Stopping rule.** The selection is repeated for all combinations of $\varepsilon \in \{5{\cdot}10^{-4}, 10^{-3}, 5{\cdot}10^{-3}, 10^{-2}\}$ and $m \in \{1, 3\}$, and every distinct selected set is trained (Section 7.1).

6. **Selection criterion.** For a fixed number $K \in \{2, 4\}$ of selected feature metapaths, the conditional criterion is compared on DBLP and IMDB with an unconditional variant, an F1-based variant, a label-free similarity criterion, the shortest metapaths and a random selection (Section 7.2).

[3] Code of this thesis: `https://github.com/nasibhuseynzade/IG-SeHGNN`.

7. **Cost of the selection.** The time of the selection step is compared with the training time of the selected model (Section 7.3).

Table 5.2: Compared methods. "Label prop." states whether the reported results use label propagation, and "Source of results" whether the results are quoted from the cited table or come from our own runs; "–" means that this is not stated in the cited source. L and L_Y are the hop budgets of feature and label metapaths.

Method	Metapaths	Label prop.	Source of results
RGCN, HetGNN, HAN, MAGNN	metapath-based (grouping of [12])	no[a]	[12, Table 3]
RSHN, HetSANN, HGT, HGB	metapath-free (grouping of [12])	no[a]	[12, Table 3]
SeHGNN (published)	configuration of [12]	yes	[12, Table 3]
LMSPS	$M = 30$ searched; maximum hop 6 / 5 / 5 (DBLP / ACM / IMDB)	no	[8, Table 1]
AMP-HGNAS	searched (metapath dependency search)	–	[1, Table 3]
SeHGNN (paper setting)	all; $L = 2/4/4$ (DBLP / ACM / IMDB), $L_Y = 4$	yes	own runs
SeHGNN (hop-4)	all; $L = L_Y = 4$	yes	own run on DBLP (identical to the paper setting on ACM and IMDB)
IG-SeHGNN (ours)	selected from the feature metapaths with $L = 4$; all label metapaths with $L_Y = 4$	yes	own runs

[a] According to [8], whose Table 1 contains the same values for these eight methods and does not use label propagation for any baseline.

Table 5.3: SeHGNN hyper-parameters that differ between the datasets, taken from the published configuration of the official implementation. All other SeHGNN hyper-parameters are the same on all datasets (Section 5.4); the selection hyper-parameters of IG-SeHGNN are listed in Table 4.3.

Hyper-parameter	DBLP	ACM	IMDB
Hop budget L of feature metapaths, SeHGNN (paper setting)	2	4	4
Hop budget L of feature metapaths, SeHGNN (hop-4) and IG-SeHGNN	4	4	4
Hop budget L_Y of label metapaths, all models	4	4	4
Layers of the downstream MLP	3	1	4
Input dropout	0.5	0.5	0
Residual connection from the target features	yes	no	no
Loss function	CE	CE	BCE

Chapter 6

Results

This chapter reports the main results of IG-SeHGNN on DBLP, ACM and IMDB; the experimental setup is described in Chapter 5. Section 6.1 verifies that our SeHGNN runs reproduce the published results, Section 6.2 compares the classification accuracy with the baselines, Section 6.3 analyses the computational cost, and Section 6.4 examines which metapaths are selected.

6.1 Reproduction of SeHGNN

Before comparing IG-SeHGNN with SeHGNN, we verified that the SeHGNN code and our environment reproduce the results published in [12]. Table 6.1 compares our runs of SeHGNN (paper setting) with the values reported in [12, Table 3].

The Micro-F1 of our runs is within 0.3 points of the published values on all three datasets; the largest difference is the Macro-F1 on IMDB (-0.88). The published results can therefore be reproduced with the available code, and the cost comparisons of Section 6.3, which rely on our own runs, are made against a faithful SeHGNN baseline.

6.2 Classification accuracy

Table 6.2 compares IG-SeHGNN in its main configuration with the baselines.

DBLP. IG-SeHGNN reaches 95.67 Micro-F1 and 95.33 Macro-F1. This is higher than the published SeHGNN result (95.42 / 95.06) [12] and than our SeHGNN run (95.35 / 94.97), on par with LMSPS (95.66 / 95.35) [8], and below AMP-HGNAS (96.11 / 95.83) [1]. IG-SeHGNN obtains this result with five feature metapaths selected from 17 candidates of up to four hops,

Table 6.1: Reproduction of SeHGNN (paper setting): test F1 (%) reported in [12, Table 3]

Dataset	Hop budget	Reported in [12] Micro / Macro	Our run Micro / Macro	Difference Micro / Macro
DBLP	2	95.42 / 95.06	95.35 / 94.97	−0.07 / −0.09
ACM	4	93.98 / 94.05	93.86 / 93.93	−0.12 / −0.12
IMDB	4	69.17 / 67.11	68.91 / 66.23	−0.26 / −0.88

Table 6.2: Test F1 (%) on the HGB datasets. Baseline results are quoted from [12, Table 3] (first nine rows), [8, Table 1] and [1, Table 3]; LMSPS is evaluated without label propagation (Section 5.2). IG-SeHGNN: main configuration, Bold: best value per column.

	DBLP		IMDB		ACM	
Method	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
RGCN	91.52±0.50	92.07±0.50	58.85±0.26	62.05±0.15	91.55±0.74	91.41±0.75
HetGNN	91.76±0.43	92.33±0.41	48.25±0.67	51.16±0.65	85.91±0.25	86.05±0.25
HAN	91.67±0.49	92.05±0.62	57.74±0.96	64.63±0.58	90.89±0.43	90.79±0.43
MAGNN	93.28±0.51	93.76±0.45	56.49±3.20	64.67±1.67	90.88±0.64	90.77±0.65
RSHN	93.34±0.58	93.81±0.55	59.85±3.21	64.22±1.03	90.50±1.51	90.32±1.54
HetSANN	78.55±2.42	80.56±1.50	49.47±1.21	57.68±0.44	90.02±0.35	89.91±0.37
HGT	93.01±0.23	93.49±0.25	63.00±1.19	67.20±0.57	91.12±0.76	91.00±0.76
HGB	94.01±0.24	94.46±0.22	63.53±1.36	67.36±0.57	93.42±0.44	93.35±0.45
SeHGNN	95.06±0.17	95.42±0.17	67.11±0.25	69.17±0.43	94.05±0.35	93.98±0.36
LMSPS	95.35±0.22	95.66±0.20	66.99±0.32	68.70±0.26	94.73±0.41	94.69±0.36
AMP-HGNAS	**95.83±0.07**	**96.11±0.06**	67.29±0.30	69.51±0.23	**95.15±0.16**	**95.13±0.16**
IG-SeHGNN (ours)	95.33±0.16	95.67±0.15	**67.47±0.23**	**69.79±0.22**	94.22±0.12	94.32±0.10

whereas SeHGNN uses all feature metapaths up to the hop budget of two of its published configuration (Table 5.3).

IMDB. IG-SeHGNN reaches 69.79 Micro-F1 and 67.47 Macro-F1 with five of 25 feature metapaths. This is 0.88 Micro-F1 and 1.24 Macro-F1 points higher than our SeHGNN run (68.91 / 66.23, Table 6.1) and also higher than the published results of SeHGNN (69.17 / 67.11) [12], LMSPS (68.70 / 66.99) [8] and AMP-HGNAS (69.51 / 67.29) [1], so that IG-SeHGNN obtains the best result of Table 6.2 on IMDB in both metrics. The margin over AMP-HGNAS (0.28 Micro-F1 and 0.18 Macro-F1), however, is of the order of the standard deviations of the two methods (0.22–0.30).

Table 6.3: Computational cost (same machine). For DBLP, SeHGNN is reported with the hop budget of its paper setting (2) and with the hop budget of four from which IG-SeHGNN selects; on ACM and IMDB the paper setting already uses a hop budget of four. The last three columns compare IG-SeHGNN with the SeHGNN configuration of the same row.

Dataset	Model	k	#Params	Time / epoch (s)	Speed-up of IG-SeHGNN	Params of IG-SeHGNN	$(k/k')^2$
DBLP	SeHGNN, hop 2 (paper setting)	9	10.84M	0.359	1.04×	+1.5%	1.0
DBLP	SeHGNN, hop 4	21	27.44M	0.794	**2.29**×	**−60**%	5.4
DBLP	**IG-SeHGNN**	**9**	**11.00M**	**0.346**	–	–	–
ACM	SeHGNN, hop 4 (paper setting)	61	88.51M	2.419	**3.43**×	**−74**%	6.5
ACM	**IG-SeHGNN**	**24**	**23.27M**	**0.705**	–	–	–
IMDB	SeHGNN, hop 4 (paper setting)	37	83.66M	3.119	**2.65**×	**−66**%	4.7
IMDB	**IG-SeHGNN**	**17**	**28.16M**	**1.176**	–	–	–

ACM. On ACM, IG-SeHGNN reaches 94.32 Micro-F1 and 94.22 Macro-F1. This is higher than the published SeHGNN result (93.98 / 94.05) [12] and than our SeHGNN run (93.86 / 93.93), by 0.46 Micro-F1 and 0.29 Macro-F1 points in the latter case, but below LMSPS (94.69 / 94.73) [8] and AMP-HGNAS (95.13 / 95.15) [1]. The selection is most aggressive for ACM, in which just four of 41 feature metapaths are retained (Table 6.4). However, the ablation in Section 7.1 demonstrates the sensitivity of the selection on the probe—the Micro-F1 on linear probe selection to fit the high-dimensional probes of this dataset does not reach the level of ACM (92.5–92.6), and a selection with another probe initialisation and no PSP still achieves 92.5–92.6 Micro-F1.

6.3 Computational cost

Table 6.3 compares the number of channels k, the number of parameters and the training time per epoch of SeHGNN and IG-SeHGNN, measured under identical conditions.

Parameters and time per epoch. On ACM and IMDB, where the paper setting of SeHGNN already uses a hop budget of four, IG-SeHGNN reduces the number of channels from 61 to 24 and from 37 to 17. This removes 74% and 66% of the parameters and makes each training epoch 3.4 and 2.7 times faster. The time measurements on the CPU are noisy: in an earlier run on IMDB the speed-up per epoch was 3.2× (2.409 s vs. 0.762 s). The reduction follows the complexity of SeHGNN, $\mathcal{O}(nd(k^2 + kd))$ per epoch [12, Table 5]: the attention term shrinks by $(k/k')^2$, i.e. by 6.5 on ACM and 4.7 on IMDB, and the projection term by k/k'. Because the projection term and data handling scale only linearly with k, the measured speed-up is smaller

than $(k/k')^2$. The parameter reduction is larger than the reduction of k alone, because the removed feature channels also remove their input matrices, whose size grows with the dimension of the raw features (Eq. (9.10)).

DBLP. On DBLP, the advantage of IG-SeHGNN is not so much the lower cost, but the higher accuracy at the same cost (+0.32 Micro-F1 and +0.36 Macro-F1 are achieved for our SeHGNN run, which searches up to 4 hops, but keeps only the useful longer metapaths). Compared to SeHGNN, which uses the same four-hop pool, IG-SeHGNN is 2.3 times faster per epoch with 60% fewer parameters.

End-to-end cost. The above are per training epoch. The selection itself is performed once at the beginning of the training and it costs in itself (Section 7.3). The selection of which greedy steps to take took 30 s on DBLP and 317 s on IMDB, but four greedy steps were counted, as later steps select probes with more channels and thus took about twice as long. The cost of the selection is hence higher than the cost of the training of the selected model (as shown in IMDB, 235 s for 200 epochs at 1.176 s per epoch), and for a training run, the end-to-end training time for IG-SeHGNN is not less than the end-to-end training time of SeHGNN (as shown in IMDB, 620 s for 200 epochs at 3.119 s per epoch). The selection does not depend on the training seed and is only executed one time and shared to all training runs (e.g. SeHGNN requires about 3,100 s for training on IMDB while IG-SeHGNN requires only one selection plus about 1,200 s of training). This is not a trade-off for the reduction in parameters and hence the reduction in memory. Measuring the cost of the selection is discussed in Chapter 8 along with ways to minimize it.

6.4 Selected metapaths

Table 6.4 lists the feature metapaths used by SeHGNN and those selected by IG-SeHGNN in the main configuration; Figure 6.1 visualises the selection on IMDB. The label metapaths are identical for both models. In ACM, S denotes subject nodes; the SeHGNN code labels them `C` (e.g. the code key `PCP` corresponds to PSP).

Observed patterns.

- **DBLP.** Each of the selected non-trivial metapaths starts in a term (T) or venue (V) node. The selection restricts the metapaths used

Table 6.4: Feature metapaths used by SeHGNN (paper setting) and selected by IG-SeHGNN (main configuration).

Dataset (target)	SeHGNN (paper setting)	IG-SeHGNN: selected from the hop-4 pool	Depths	Label metapaths (both models)
DBLP (author)	A, AP, APA, APT, APV (all 5 of hop ≤ 2)	A, APTP, APTPA, APVPA, APVPT (5 of 17)	0, 3, 4, 4, 4	APA, APAPA, APTPA, APVPA (4)
ACM (paper)	all 41 of hop ≤ 4	P, PSP, PSPP, PAPPP (4 of 41)	0, 2, 3, 4	20 closed metapaths
IMDB (movie)	all 25 of hop ≤ 4	M, MK, MDMK, MKMDM, MAMKM (5 of 25)	0, 1, 3, 4, 4	12 closed metapaths

by SeHGNN (short metapaths: AP, APA, APT, APV, etc.) to three and four hop metapaths (APVPA (authors publishing in the same venue), APTPA (authors using the same terms), etc.) and removes the metapaths starting with the connection between the author and the paper (APAP... etc.) found redundant by the conditional criterion.

- **ACM.** Two of the three chosen non-trivial metapaths go through subject nodes: PSP is a paper on the same subject, PSPP is a paper on the same subject added with one paper–paper link, PAPPP is a paper one co-author and two paper–paper links removed from the target paper. The implementation is based on the combined citations/references relation, Section 5.1. This is in line with the task, that has three classes, which are research areas. PSP is also one of the two metapaths which are used by HAN as hand-designed input on ACM [7].

- **IMDB.** The four metapaths that are selected are all non-trivial metapaths that contain the keyword node, K (MK, MDMK, MKMDM, MAMKM). Metapaths through which only actors or directors occur (MAM, MDM) are not chosen as feature channels, but are left as label channels for the model.

- **Automatic depth.** For all three datasets the selected metapaths are of mixed lengths varying from the node features (0 hops) up to four hops and the datasets have no hop budget defined. Several short metapath are discarded and longer ones are preserved (for instance, no 1 or 2 hop metapath are selected on DBLP).

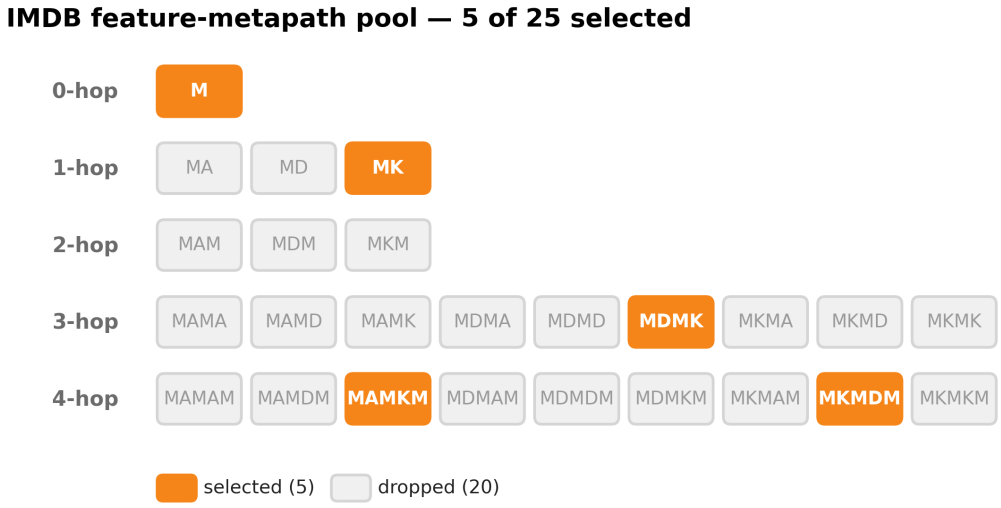


Figure 6.1: Feature-metapath candidate pool of IMDB (25 metapaths of up to four hops) and the five metapaths kept by IG-SeHGNN in the main configuration.

6.5 Summary

IG-SeHGNN is the most accurate one on all the three datasets and achieves the best result on IMDB; it is on par with LMSPS on DBLP and still worse than LMSPS on DBLP and ACM. It retains 4–5 out of 17–41 candidate feature metapaths. This decreases the parameters by 66–74% and speeds up each training epoch by 2.7–3.4 times on ACM and IMDB, respectively, and leads to higher accuracy on DBLP with the same costs as the paper setting of SeHGNN. The metapaths selected are interpretable, and are dependent on the dataset: term or venue paths on DBLP, subject paths on ACM or keyword paths on IMDB. The accuracy, for ACM, selected set, depends on the initialisation of the probe (Section 7.1). The selection itself is a cost, which reduces the end-to-end gain and is analysed, in Chapter 7.

Chapter 7

Ablation Studies

This chapter investigates the two key design decisions of IG-SeHGNN: the stopping criterion for deciding when to stop generating metapaths (Section 7.1) as well as the task-aware conditional selection criterion (Section 7.2). The cost of the selection itself is reported in §7.3.

For all ablations, the data, candidate pools, and SeHGNN hyper-parameters and training procedure are the same as Chapter 5, using [five] training seeds for each configuration. To compare stopping rules on equal terms, the greedy selection path of each dataset is computed once for the 12 stopping rules with a fixed probe seed, and the stopping rule of Section 4.6 is applied to this path for every setting. Only training selected sets that are distinct from others.

The main configuration used in Chapter 6 was run without using a fixed probe seed (as described in Section 5.4), and the paths of the fixed-seed configurations do not reflect the choices of the main configuration precisely. The third and fourth metapaths are different in all runs on DBLP, and the second and third metapaths are different for all runs on ACM. The first four metapaths coincide on IMDB, while the main run stops after these four metapaths, the fixed-seed run maintains a fifth one (MKMK) at the same threshold $\varepsilon = 10^{-3}$ (Table 7.3). The difference has an impact on precision. The accuracy of the fixed-seed paths (Table 6.2) is 94.96, 92.49 and 68.43 Micro-F1 on DBLP, ACM and IMDB, respectively, which is better than any setting in Tables 7.1–7.3 yet not as good as the accuracy of the main configuration (95.67, 94.32 and 69.79 Micro-F1 on DBLP, ACM and IMDB, respectively). The latter stages of the greedy selection, and thus the accuracy, are thus dependent on initialisation of the probe (Finding 4 in Section 7.1). The settings on the ablations are to be read as relative comparisons and ablate along one fixed path per set of data. Selectors should be evaluated using a fixed (or set) probe seed or be evaluated using multiple probe seeds during the final evaluation.

Table 7.1: DBLP. Reference: SeHGNN with the hop budget of its paper setting, $k = 9$, 10.8M parameters, 0.359 s per epoch, 95.35 / 94.97.

ε	m	#sel	Selected metapaths	k	Params	s / epoch	Micro / Macro
0.0005	1, 3	4	APVPA, APTPA, APTPV, APVP	9	11.0M	0.339	**95.63 / 95.31**
0.001	1, 3	3	APVPA, APTPA, APTPV	8	8.0M	0.362	94.96 / 94.56
≥ 0.005	3	3	APVPA, APTPA, APTPV	8	8.0M	0.362	94.96 / 94.56
≥ 0.005	1	2	APVPA, APTPA	7	7.2M	0.269	95.39 / 94.99

Table 7.2: ACM. Reference: SeHGNN, $k = 61$, 88.5M parameters, 2.419 s per epoch, 93.86 / 93.93.

ε	m	#sel	Selected metapaths	k	Params	s / epoch	Micro / Macro
all	1	2	PAPPP, PSPPP	23	21.5M	0.523	**92.59 / 92.64**
all	3	3	PAPPP, PSPPP, PAPSP	24	23.3M	0.667	92.49 / 92.58

7.1 Stopping threshold and minimum number of metapaths

The improvement threshold ε and minimum number m (Section 4.6) control the number of selected metapaths. We vary $\varepsilon \in \{0.0005, 0.001, 0.005, 0.01\}$ and $m \in \{1, 3\}$ with patience $\pi = 2$ and $M = 12$. The settings that result in the same set of selected feature metapaths are combined in Tables 7.1–7.3, with the number of selected feature metapaths being denoted by "#sel" and the target node's own features also being added to this set.

Findings.

1. **On DBLP and IMDB, only a few number of metapaths is enough.** Setting $\varepsilon = 0.005$ and $m = 1$ results in just two feature metapaths being present in each dataset. On IMDB this is the most accurate setting of Table 7.3: with two of 25 feature metapaths it matches our SeHGNN run, with 78% less parameters and 5.6× faster epoch. The four metapaths of the main configuration are more accurate (69.79 / 67.47, Table 6.2), and save less time (66% fewer parameters, 2.7× faster epoch). On DBLP, the two metapaths achieve the same paper accuracy (95.39 vs. 95.35 Micro-F1) with 34% less number of parameters and 1.3× faster epoch; the most accurate setting for DBLP contains 4 metapaths (95.63 / 95.31 Micro-F1). On ACM two metapaths are not enough on this path (Finding 4).

2. **The information gain falls off rapidly.** As is seen in Figure 7.1,

Table 7.3: IMDB. Reference: SeHGNN, $k = 37$, 83.7M parameters, 3.119 s per epoch, 68.91 / 66.23.

ε	m	#sel	Selected metapaths	k	Params	s / epoch	Micro / Macro
0.0005	1, 3	6	MKMDM, MAMKM, MK, MDMK, MKMK, MAMK	19	37.9M	0.983	68.61 / 66.21
0.001	1, 3	5	MKMDM, MAMKM, MK, MDMK, MKMK	18	33.0M	0.796	68.43 / 66.10
≥ 0.005	3	3	MKMDM, MAMKM, MK	16	23.3M	0.869	68.37 / 65.93
≥ 0.005	1	2	MKMDM, MAMKM	15	18.4M	0.555	**68.96 / 66.54**

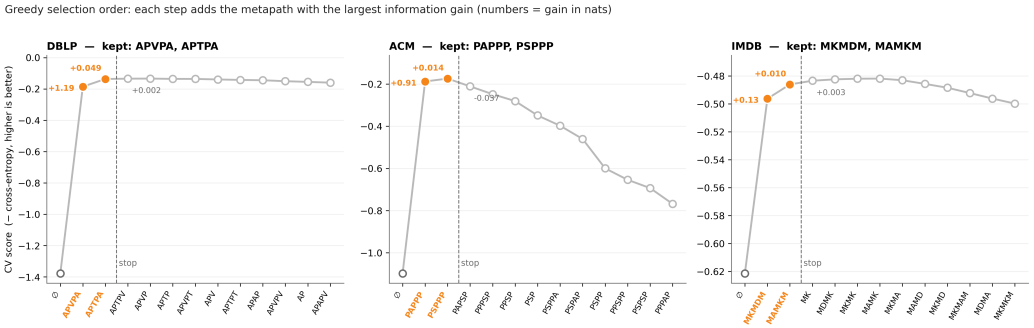


Figure 7.1: Greedy selection paths: cross-validated score after each greedy step and the estimated information gain of the first steps; the metapaths kept with $\varepsilon = 0.005$, $m = 1$ are highlighted.

the first selected metapath contains almost all the information that the probe can find (1.19 nats on DBLP, 0.91 nats on ACM, 0.13 nats on IMDB), and later metapaths only add 0.05 nats or less, with the third or fourth step adding at most 0.003 nats.

3. **The minimum $m = 3$ does not help.** Along the fixed-seed paths, the additional metapath that the minimum forces into the set is neutral or harmful on every dataset (IMDB Macro-F1 66.54 → 65.93, DBLP Micro-F1 95.39 → 94.96, ACM 92.59 → 92.49). A minimum of one metapath is therefore preferable.

4. **On ACM the probe, not the threshold, limits the selection.** It is possible to see that after the second metapath, each step for every ε has a negative estimated gain, so that the estimated gain decreases along the trail. The cross-validated score drops precipitously with each additional channel (between −0.19 and −0.77 nats in twelve steps, Figure 7.1). This kind of behavior does not imply that the other metapaths are uninformative, but rather that the linear probe over-fits the high dimensional channels in ACM (1,902 dimensions each) (Section 4.5). This means that the decision in the second step is based on small or

negative estimated gains. However, the fixed-seed path is unable to reach PSP in any step, and only in the sixth step with a negative estimated gain; the selected sets are 92.5–92.6 Micro-F1, that is only 1.3 points less than in our SeHGNN run and 1.7–1.8 points less than in the main configuration, whose selection includes PSP and PSPP (Section 6.2). The accuracy of IG-SeHGNN on ACM is thus dependent of the initialisation of the probe.

7.2 Selection criterion

This ablation will assess if the accuracy of IG-SeHGNN is because of *which* metapaths the task-aware criterion chooses or is because of the less number of channels, and whether the *conditional* form of the criterion is important.

Protocol. For each method, we sample the same number K of feature metapaths from the same candidate pool of metapaths ($K = 16$ on DBLP and $K = 24$ on IMDB, where the target's channel is excluded from the candidate pool). For every method, the channels are all the target's own features and the family of label metapaths (all the configurations with the same K have the same number of channels). The chosen ones are used for training the unaltered SeHGNN. Two budgets are used, $K = 2$ and $K = 4$. The methods compared are summarised in Table 7.4. The CKA method is a label-free proxy for structural redundancy criteria: it discards channels which look similar, without taking the usefulness of channels into consideration.

Findings.

1. **With a limited number of metapaths on IMDB, the most important factor is task-awareness.** With $K = 2$, the three label-aware options (ours, unconditional, F1) achieve Micro-F1 of 68.8–69.0 whereas the three label-free options (CKA, shortest paths, random) achieve Micro-F1 of 68.0–68.1, which is only about 0.9 points worse. This is more than twice as big as the 5% of the random baseline, and our choice is better than all the 5 randomly drawn samples! Label-free redundancy removal (CKA) is as good as random: it retains channels that are different, but do not contain much information on the genres. This is not the case in DBLP: While most of the information is already captured by the label metapaths (Finding 4), the label-free CKA selection attains the highest value at $K = 2$ (95.67 / 95.33).

Table 7.4: Compared selection methods.

Method	How the K metapaths are chosen	Uses labels	Redundancy-aware
Ours (conditional information gain)	greedy maximisation of $\widehat{\mathrm{IG}}(\mathcal{P} \mid S)$ (Eq. (4.15))	yes	yes
Unconditional information gain	each metapath scored on its own, $\widehat{\mathrm{IG}}(\mathcal{P} \mid \emptyset)$; top K	yes	no
F1 criterion	same greedy procedure, scored by cross-validated F1 instead of cross-entropy	yes	yes
CKA (label-free)	start from the target channel; repeatedly add the channel least similar (linear CKA on 2,000 nodes) to those selected	no	similarity only
Shortest paths	the K shortest metapaths; for $K = 4$ on DBLP this is the paper setting of SeHGNN	no	no
Random	K metapaths drawn at random; 5 draws, mean $\pm$ standard deviation	no	no

2. **Conditioning rules out repetitive choices.** The best metapaths in isolation are close to being replicas: all of the 6 best metapaths on DBLP pass through the venue node, and all of the 6 best metapaths on IMDB contain the keyword node. This means that, the unconditional criterion spends its budget by filling its metapaths with overlapping ones and at $K = 4$ it is below our conditional criterion on both data sets (DBLP -0.49; IMDB -1.31 Micro-F1). The conditional criterion, however, includes complementary metapaths, and thus includes the term path APTPA second, which is the seventh ranked term on its own.

3. **At $K = 2$, cross-entropy and F1 are comparable.** At $K = 2$, there is no consistent improvement in accuracy for either scoring function. The cross-entropy score is smoother and the F1 criterion diverges from it early on IMDB where the multi-label F1 is a coarse signal and at

Table 7.5: DBLP: test Micro-F1 / Macro-F1 (%).

Method	$K = 2$	$K = 4$	Selected at $K = 4$	Selection time
Ours	95.39 / 94.99	**95.63** / [95.31]	APVPA, APTPA, APTPV, APVP	30.0 s
Unconditional	95.32 / 94.96	95.14 / 94.74	APVPA, APAPV, APV, APVPV	6.4 s
F1 criterion	95.39 / 94.99	95.56 / 95.22	APVPA, APTPA, APTPV, APVPV	30.9 s
CKA (label-free)	**95.67** / **95.33**	95.60 / 95.26	APAPT, APV, APAP, APTPA	1.1 s
Shortest paths	95.49 / 95.11	95.35 / 94.97	AP, APA, APT, APV	–
Random (5 draws)	95.15±0.32 / 94.75±0.35	95.54±0.13 / 95.20±0.14	–	–

Table 7.6: IMDB: test Micro-F1 / Macro-F1 (%). Reference: SeHGNN with all 25 feature metapaths, 68.91 / 66.23.

Method	$K = 2$	$K = 4$	Selected at $K = 4$	Selection time
Ours	68.96 / 66.54	**69.79** / **67.47**	MKMDM, MAMKM, MK, MDMK	316.8 s
Unconditional	68.77 / **66.65**	68.48 / 65.94	MKMDM, MKM, MKMKM, MKMAM	35.5 s
F1 criterion	**69.04** / 66.44	68.51 / 66.24	MKM, MAMDM, MDMDM, MAMKM	288.3 s
CKA (label-free)	67.99 / 65.84	69.04 / 66.46	MAMD, MKMK, MDMAM, MKMAM	2.5 s
Shortest paths	68.13 / 65.52	69.02 / 65.77	MA, MD, MK, MAM	–
Random (5 draws)	68.00±0.37 / 65.62±0.41	68.69±0.30 / 66.13±0.43	–	–

$K = 4$ is 1.28 Micro-F1 below the cross-entropy criterion on IMDB.

4. **The methods converge on DBLP and the conditional criterion remains in the lead on IMDB.** On DBLP, all the twenty trained configurations are in the range around one point (94.75–95.77) since the four label metapaths include most of the information, thus giving the model sufficient information regardless of which feature metapaths were chosen. In contrast, the best method at $K = 4$ on IMDB is the conditional criterion, with scores of 69.79 / 67.47 Micro-F1 / Macro-F1, while other methods score between 68.48 and 69.04 Micro-F1, and between 65.77 and 66.46 Macro-F1. Its margin is 0.75 Micro-F1 and 1.01 Macro-F1 better than the next best method (CKA) and 1.10 / 1.34 better than the mean of the random draws (more than three standard deviations of the random baseline). The value of the criterion then indicates where the label metapaths don't already suffice—on IMDB it finds a very small sufficient set at $K = 2$ and the most accurate set at $K = 4$.

7.3 Cost of the selection

Times were recorded for four greedy steps in making selection in Tables 7.5, 7.6. The conditional criterion is the most expensive as all steps require as many probes as there are candidates (and folds) left: 30 s on DBLP (featuring 16 candidates and 5 folds) and 317 s on IMDB (24 candidates and 5 folds – larger channels, multi-label targets). For comparison, it takes approximately 70 s

to train the selected model on DBLP for 200 epochs and approximately 110–235 s to train on IMDB, also depending on the number of selected channels. The fact that the selection is independent of the training seed means that its usage cost is only incurred once, and is amortized if multiple trainings (e.g. on multiple seeds, or during hyperparameter tuning) occur. At $K = 2$, the unconditional criterion achieves similar accuracy at IMDB but with a nine times cheaper selection (35.5 s vs. 68.48 s). At $K = 4$, the conditional and unconditional criteria are 1.31 Micro-F1 apart (68.48 Micro-F1 vs. 69.79 Micro-F1). This implies that it could be used to order the candidates prior to the conditional search, instead of replacing the search.

7.4 Summary

The ablations show that (i) two to four well-chosen feature metapaths are sufficient to match or outperform SeHGNN on DBLP and IMDB, (ii) on the fixed-seed paths we do not need $m = 3$, (iii) on ACM the linear probe over-fits, and thus early stops selection resulting in its accuracy dependent upon the initialisation of the probe (92.5–92.6 Micro-F1), (iv) on IMDB, conditioning on the already selected metapaths prevents redundant choices and leads to the most accurate set at $K = 4$, while on DBLP all conditions perform similarly and (v) the selection cost is not insignificant and must be reduced for end-to-end efficiency on these small datasets.

Chapter 8

Conclusion

8.1 Summary

This thesis focused on asking itself the question of which metapaths to use for a heterogeneous graph neural network. The Simple and Efficient Heterogeneous Graph Neural Network (SeHGNN) [12] pre-computes each metapath into a channel, and then fuses all channels via a transformer. In its report, it will be based on all metapaths in a fixed hop budget per data set and the fusion cost increases quadratically with the number of metapaths [12]. To this end, in this thesis, based on the information-gain based meta-path learning in MP-GNN [10], a task-aware and automatic metapath selection step was proposed to be performed prior to SeHGNN without altering its architecture. The score of each candidate metapath is measured by the held-out cross-entropy reduction of a small probe given metapath added to the already selected metapaths, an estimation of the conditional mutual information between the metapath and the task labels. Metapaths are greedily added until this gain is close to 0, in which way redundant metapaths can be removed and the metapath depth is defined by the data.

8.2 Main findings

- **Less metapaths with no loss of accuracy in two out of three datasets.** IG-SeHGNN performs a minimum 4–5 of 17–41 candidate feature metapaths reduction. On DBLP and IMDB it is more accurate than SeHGNN (95.67 vs. 95.42 Micro-F1) and on par with LMSPS, on IMDB it is on par with our SeHGNN run (69.79 vs. 68.91 Micro-F1). On ACM, it is roughly 0.8 points less than SeHGNN.

- **Lower cost.** For the main configuration, the number of parameters is reduced by 66–74% on ACM and IMDB, while each iteration of the training is reduced by a factor of 2.7–3.4, similar to a quadratic dependency of the cost of SeHGNN on the number of channels [12]. IG-SeHGNN is able to match the performance of SeHGNN on IMDB with only 78% of the parameters and 5.6 times faster epoch, with just two selected feature metapaths ($\varepsilon \geq 0.005$, $m = 1$; Section 7.1). On DBLP, IG-SeHGNN achieves a higher accuracy at the same cost as the accuracy that the paper setting of SeHGNN achieves.

- **Automatic depth.** The chosen metapaths have various depths (from 0 to 4 hops) and no hop budget had to be selected per dataset.

- **Task-awareness matters.** Label-aware selection with a small budget performs about 0.9 F1 point better than label-free selections (random, shortest paths, similarity-based redundancy removal), and using the already reresected metapaths as conditions prevents picking up near-duplicate metapaths.

- **Interpretability.** Selected metapaths are: term path and venue path for research areas on DBLP, subject path on ACM and keyword path on IMDB.

8.3 Limitations

- **Probe.** The linear probe over-fits high-dimensional channels on ACM, which ends the selection too early and causes the accuracy gap on this dataset.

- **The range of the selection.** The only metapaths which are selected are feature metapaths; label metapaths, of course, are never selected; the probe does not condition on label metapaths. Budget of all candidates up to the hop still needs to be done.

- **Reproducibility.** A fixed seed of the probe is required because the probe initialisation is done at random and the choices of the greedy path may differ from one run to another.

- **Greedy search.** There is no optimality guarantee for forward selection.

8.4 Future work

- **A better probe.** A more stringent regularisation, dimensionality reduction of the channels or a non-linear probe head could alleviate this over-fitting seen on ACM.
- **Label-side selection.** Extend the selection to label metapaths, with fold-specific label propagation to avoid leakage.
- **Greater hop budgets and greater graphs.** On ACM and IMDB, the maximum depth of selected metapaths is four hops, indicating that larger budgets might lead to better performance; larger graphs with significantly more candidate metapaths (e.g., ogbn-mag) are the ones where selection should have the biggest impact.

Chapter 9

Appendix

This appendix collects the derivations and details that support Chapter 4: the information-theoretic definitions (Section 9.1), a worked example (Section 9.2), the relation between the probe loss and the conditional entropy (Section 9.3), the multi-label probe (Section 9.4), the parameter count and the selection cost (Section 9.5), implementation details of the SeHGNN backbone (Section 9.6), the scoring function of MP-GNN (Section 9.7) and the implementation files

9.1 Information-theoretic definitions

The following definitions are the standard ones of information theory. Natural logarithms are used, so all quantities are measured in nats. Y denotes the label of a randomly drawn labelled node and X a channel vector.

Entropy. The entropy of the label measures how uncertain the label is before any information about the node is observed:

$$H(Y) = -\sum_{y \in \mathcal{Y}} p(y) \ln p(y), \tag{9.1}$$

where $\mathcal{Y}$ is the set of classes and $p(y)$ the probability of class y. The entropy is zero when the label is always the same and reaches its maximum $\ln|\mathcal{Y}|$ when all classes are equally likely; a balanced binary task has $H(Y) = \ln 2 \approx 0.693$ nats.

Conditional entropy. The uncertainty that remains about the label after X has been observed is

$$H(Y \mid X) = -\sum_{x} p(x) \sum_{y} p(y \mid x) \ln p(y \mid x) = -\,\mathbb{E}_{(x,y)}[\ln p(y \mid x)]. \quad (9.2)$$

Observing a variable never increases the uncertainty on average, so $0 \le H(Y \mid X) \le H(Y)$.

Mutual information. The reduction of uncertainty about Y obtained by observing X is

$$I(X;Y) = H(Y) - H(Y \mid X) = \sum_{x,y} p(x,y) \ln \frac{p(x,y)}{p(x)\,p(y)}. \quad (9.3)$$

It is symmetric, non-negative, and zero if and only if X and Y are independent.

Conditional mutual information and chain rule. The conditional mutual information $I(X^{\mathcal{P}};Y \mid X^{S})$ is defined in Eq. (4.9). The information carried by a set of channels decomposes by the chain rule

$$I(X^{S}, X^{\mathcal{P}}; Y) = I(X^{S};Y) + I(X^{\mathcal{P}};Y \mid X^{S}), \quad (9.4)$$

and repeated application gives Eq. (4.10). All these quantities are bounded:

$$0 \;\le\; I(X^{\mathcal{P}};Y \mid X^{S}) \;\le\; H(Y \mid X^{S}) \;\le\; H(Y) \;\le\; \ln|\mathcal{Y}|. \quad (9.5)$$

9.2 A worked example

The following toy example shows why the *conditional* form of the criterion matters. Consider eight labelled movies with two balanced classes (T = thriller, C = comedy) and three candidate metapath channels, each reduced to a binary value for simplicity (Table 9.1).

Label entropy. Four T and four C give $H(Y) = \ln 2 = 0.693$ nats.

Channel $\mathcal{P}_1$ alone. Group a contains three T and one C, group b one T and three C, so

$$H(Y \mid X^{\mathcal{P}_1}) = -(\tfrac{3}{4}\ln\tfrac{3}{4} + \tfrac{1}{4}\ln\tfrac{1}{4}) = 0.562, \qquad I(X^{\mathcal{P}_1};Y) = 0.693 - 0.562 = 0.131.$$

Channel $\mathcal{P}_2$ alone. It is a copy of $\mathcal{P}_1$, so $I(X^{\mathcal{P}_2};Y) = 0.131$ as well.

Table 9.1: Toy example with eight labelled nodes and three candidate channels.

Node	1	2	3	4	5	6	7	8
Label Y	T	T	T	C	T	C	C	C
$X^{\mathcal{P}_1}$	a	a	a	a	b	b	b	b
$X^{\mathcal{P}_2}$ (copy of $X^{\mathcal{P}_1}$)	a	a	a	a	b	b	b	b
$X^{\mathcal{P}_3}$	v	v	v	u	u	v	v	v

Channel $\mathcal{P}_3$ alone. Group u contains one T and one C, group v three T and three C, so $H(Y \mid X^{\mathcal{P}_3}) = \ln 2$ and $I(X^{\mathcal{P}_3}; Y) = 0$: on its own, $\mathcal{P}_3$ is useless.

Conditional gains after selecting $\mathcal{P}_1$. Since $\mathcal{P}_2$ repeats $\mathcal{P}_1$, $I(X^{\mathcal{P}_2}; Y \mid X^{\mathcal{P}_1}) = 0$. In contrast, the pair $(X^{\mathcal{P}_1}, X^{\mathcal{P}_3})$ determines the label exactly, namely $(a, v) \to$ T, $(a, u) \to$ C, $(b, u) \to$ T and $(b, v) \to$ C, so $H(Y \mid X^{\mathcal{P}_1}, X^{\mathcal{P}_3}) = 0$ and

$$I(X^{\mathcal{P}_3}; Y \mid X^{\mathcal{P}_1}) = 0.562 - 0 = 0.562.$$

Consequence. Ranking the channels by their individual information gain selects $\{\mathcal{P}_1, \mathcal{P}_2\}$, which together carry only 0.131 nats. Selecting by conditional information gain chooses $\mathcal{P}_1$ and then $\mathcal{P}_3$, which together carry $0.131 + 0.562 = 0.693$ nats, i.e. all the information about the label. The conditional criterion thus rejects the redundant channel and finds the complementary one.

9.3 Cross-entropy as an upper bound of the conditional entropy

The expected cross-entropy of a probe q_S under the true data distribution p is $\mathrm{CE}(q_S) = \mathbb{E}_{(x,y)\sim p}[-\ln q_S(y \mid x)]$. Adding and subtracting $\ln p(y \mid x)$ inside the expectation gives

$$-\ln q_S(y \mid x) = -\ln p(y \mid x) + \ln \frac{p(y \mid x)}{q_S(y \mid x)}. \tag{9.6}$$

The expectation of the first term is $H(Y \mid X^S)$ by Eq. (9.2). The expectation of the second term is

$$\mathbb{E}_x[\sum_y p(y \mid x) \ln \frac{p(y \mid x)}{q_S(y \mid x)}] = \mathbb{E}_x[\mathrm{KL}(p(\cdot \mid x) \,\|\, q_S(\cdot \mid x))] = \Delta(S) \;\geq\; 0, \tag{9.7}$$

which proves Eq. (4.13).

Decomposition of the estimated gain. Substituting Eq. (4.13) for S and for $S \cup \{\mathcal{P}\}$ into Eq. (4.15) gives

$$\widehat{\mathrm{IG}}(\mathcal{P} \mid S) = \underbrace{H(Y \mid X^S) - H(Y \mid X^S, X^{\mathcal{P}})}_{= I(X^{\mathcal{P}}; Y \mid X^S)} + \underbrace{\Delta(S) - \Delta(S \cup \{\mathcal{P}\})}_{\text{difference of approximation gaps}} + \xi, \tag{9.8}$$

where ξ is the sampling error of the held-out estimate. The estimate is therefore close to the conditional mutual information when the approximation gap of the probe changes little as $\mathcal{P}$ is added and when enough held-out nodes are available. A negative estimate means that $\Delta(S \cup \{\mathcal{P}\})$ is larger than $\Delta(S)$, i.e. that the probe over-fits the new channel. Summing Eq. (4.15) over a sequence of selected channels telescopes to $\mathrm{Score}(S_m) - \mathrm{Score}(\emptyset)$, which by Eq. (4.10) estimates $I(X^{S_m}; Y) \le H(Y)$.

9.4 Multi-label probe

For the multi-label dataset IMDB, the probe outputs one logit s_c per class and models each label independently with a sigmoid, $q_S(\mathbf{y} \mid \mathbf{x}) = \prod_{c=1}^{C} \sigma(s_c)^{y_c} (1 - \sigma(s_c))^{1-y_c}$. The loss on fold t is the binary cross-entropy, averaged over classes:

$$\mathrm{BCE}_{V_t}(q_S) = -\frac{1}{|V_t|\, C} \sum_{i \in V_t} \sum_{c=1}^{C} [y_{ic} \ln \sigma(s_{ic}) + (1 - y_{ic}) \ln (1 - \sigma(s_{ic}))]. \tag{9.9}$$

Up to the factor $1/C$, this is the cross-entropy of the factorised model, so the argument of Section 9.3 applies unchanged: the score is bounded by the conditional entropy of the label vector, and the gain estimates the conditional mutual information in nats per class. The empty-set probe predicts the per-class label frequencies.

9.5 Parameter count and selection cost

Parameters. From the implementation of the backbone (Section 9.6), the number of parameters can be written approximately as

$$|\theta| \approx d_e \sum_{\mathcal{P} \in \Phi_X^*} d_{\mathcal{P}} + d_e\, C\, |\Phi_Y| + k(d_e\, d + (n_{fp} - 1)\, d^2 + d^2) + \mathcal{O}(d^2 + d_e d), \tag{9.10}$$

where n_{fp} is the number of feature-projection layers. The first term contains the input matrices of the feature channels, the second those of the label channels, the third the per-channel projection weights and the slice of the layer after the concatenation, and the last term the parameters that do not depend on the channels (semantic fusion, residual connection and task MLP); biases and normalisation parameters are omitted. On IMDB with $d_e = d = 512$ and $n_{fp} = 2$, for example, the input matrices of the 25 candidate feature channels alone account for about 53M of SeHGNN's 83.7M parameters, and the model restricted to the three selected feature channels has 18.4M parameters.

Selection cost. Let $J \leq M$ be the number of greedy steps that are executed. Including the empty set, the number of probe fits is

$$N_{\text{fit}} = F(1 + \sum_{j=1}^{J} (|\Phi_X| - j + 1)) \;\leq\; F\,(1 + J\,|\Phi_X|). \tag{9.11}$$

Each fit performs T full-batch optimisation steps. Let $D_S = \sum_{\mathcal{Q} \in S} d_{\mathcal{Q}}$ be the total dimension of the at most J channels of a probe; one optimisation step of the linear probe then costs $\mathcal{O}(|V_L|\, C\, D_S)$. The total selection cost therefore grows linearly with the number of candidates, the number of folds and the number of probe steps, and roughly quadratically with the number of greedy steps.

9.6 Implementation details of the SeHGNN backbone

The following details refer to the public implementation of SeHGNN [12], which is used unchanged in this thesis. Every feature channel is first multiplied by a channel-specific input matrix $E_{\mathcal{P}} \in \mathbb{R}^{d_{\mathcal{P}} \times d_e}$, and every label channel by a matrix of size $C \times d_e$. The feature projection consists of channel-specific linear layers with layer normalisation, PReLU and dropout. In the semantic fusion, queries and keys have dimension $d/4$, the attention scores are scaled by the square root of this dimension, and β is initialised to zero. The concatenated vector of size $k \cdot d$ is reduced to d dimensions by a linear layer; optionally, a residual connection from the target node's own (embedded) features is added. The multi-label dataset IMDB is trained with the binary cross-entropy loss instead of the cross-entropy loss.

9.7 Scoring function of MP-GNN

In the first iteration, MP-GNN predicts the label of node i from a candidate relation r as

$$\tilde{y}_i^r = \Theta^T h_i^{(0)} \cdot \max_{j \in \mathcal{N}_i^r} w_j, \tag{9.12}$$

where Θ is a learnable weight vector that accounts for the node features and w_j is a learnable node weight that is close to 1 if node j is predicted to belong to the ground-truth meta-path and close to 0 otherwise [10]. The score of the relation is the minimal mean squared error

$$s_r = \min_{\Theta, \mathbf{w}} \frac{1}{N} \sum_{i=1}^{N} (\tilde{y}_i^r - y_i)^2, \tag{9.13}$$

and the relation with the minimum score is selected as the first relation of the meta-path [10]. For the next iteration, a positive node i is replaced by the bag

$$B^+(i) = \{j \in \mathcal{N}_i^r \mid \nexists k : j \in \mathcal{N}_k^r \wedge y_k = -1\}, \tag{9.14}$$

negative bags are singletons, and a new relation s is scored by its potential to predict the bag labels [10]:

$$\tilde{y}_{B(i)}^s = \max_{j \in B(i)} (\Theta^T h_j^{(0)} \cdot \max_{k \in \mathcal{N}_j^s} w_k), \tag{9.15}$$

again by minimising the mean squared error, now at the level of bag labels [10].

Bibliography

[1] X. Li, P. Liò, L. Yang, Z. Ye, and C. Peng, "Adaptive meta-path based neural network architecture search for heterogeneous graphs." SSRN. n.d.

[2] X. Lou, G. Liu, and J. Li, "ASIAM-HGNN: Automatic selection and interpretable aggregation of meta-path instances for heterogeneous graph neural network," *Computing and Informatics*, vol. 42, pp. 257–279, 2023. ResearchGate.

[3] Y. Ding, Q. Yao, H. Zhao, and T. Zhang, "DiffMG: Differentiable meta graph search for heterogeneous graph neural networks," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '21)*, 2021. arXiv.

[4] H. Peng, F. Long, and C. Ding, "Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 8, pp. 1226–1238, 2005. arXiv.

[5] Y. Wu, X. Wen, W. Chen, and Y. Mu, "Information flow optimization for adaptive neighbor selection graph embedding," *Scientific Reports*, vol. 15, 2025. Art. no. 40108, arXiv.

[6] Y. Li, Y. Jin, G. Song, Z. Zhu, C. Shi, and Y. Wang, "GraphMSE: Efficient meta-path selection in semantically aligned feature space for graph neural networks," in *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21)*, pp. 4206–4214, 2021. arXiv.

[7] X. Wang, H. Ji, C. Shi, B. Wang, P. Cui, P. Yu, and Y. Ye, "Heterogeneous graph attention network," in *Proceedings of WWW 2019*, (San Francisco, USA), 2019. arXiv.

[8] C. Li, Z. Guo, Q. He, H. Xu, and K. He, "Long-range meta-path search on large-scale heterogeneous graphs." arXiv, 2025.

[9] F. S. Khan and A. Aboulnaga, "MetaSieve: Faster relational deep learning through SQL-based metapath selection." arXiv, 2026.

[10] F. Ferrini, A. Longa, A. Passerini, and M. Jaeger, "Meta-path learning for multi-relational graph neural networks." arXiv, 2023.

[11] J. Fan, J. Yang, Z. Gu, H. Wu, D. Sun, F. Qin, and J. Wu, "Path-aware multi-scale learning for heterogeneous graph neural network," *Neural Networks*, vol. 191, 2025. Art. no. 107743, arXiv.

[12] X. Yang, M. Yan, S. Pan, X. Ye, and D. Fan, "Simple and efficient heterogeneous graph neural network," in *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI-23)*, pp. 10816–10824, 2023. arXiv.

[13] X. Wang, D. Bo, C. Shi, S. Fan, Y. Ye, and P. S. Yu, "A survey on heterogeneous graph embedding: Methods, techniques, applications and sources." arXiv, 2020.